Eric Sticchi Zambom

Gustavo Garcia Leite

Reconhecimento automático de placas automobilísticas baseando-se em Redes Neurais Convolucionais

São Paulo - Brasil

2020

Eric Sticchi Zambom

Gustavo Garcia Leite

Reconhecimento automático de placas automobilísticas baseando-se em Redes Neurais Convolucionais

Monografia apresentada na disciplina Trabalho de Conclusão de Curso, como parte dos requisitos para obtenção do título de Bacharel em Ciência da Computação.

Centro Universitário Senac - Santo Amaro
Bacharelado em Ciência da Computação

Orientador: Prof. Ms. Leonardo Massayuki Takuno

São Paulo - Brasil

2020

Agradecimentos

Primeiramente agradecemos ao nosso orientador e professor Ms. Leonardo Massayuki Takuno.

A Apple Developer Academy do Centro Universitário Senac por ter nos proporcionado um ótimo ambiente de trabalho e habilidades para nossas vidas, e a todos os nossos mentores e colegas.

Acima de tudo à nossas famílias que nos apoiaram incondicionalmente, mesmo com a situação de pandemia.

Agradecimentos especiais à todos os professores que nos ajudaram a concluir esta monografia.

Resumo

O Reconhecimento Automático de Placas Automobilísticas (ALPR, *Automatic License Plate Recognition*) tem como objetivo identificar e ler o texto de placas de um ou mais veículos em uma imagem. ALPR é um tema de estudo frequente com diversas aplicações na vida cotidiana, como fiscalização de trânsito e controle de acesso à estacionamentos. Considerando o aumento drástico de veículos e sistemas de transporte no Brasil durante o século atual, a placa do veículo automotivo pode ser considerada uma identificação pessoal, uma vez que os dados do carro estão diretamente atrelados à uma pessoa física, permitindo que empresas em parceria com o governo do Brasil acessem parte dos dados de uma pessoa a partir da placa de seu veículo. Esse aumento do número de veículos dificulta o gerenciamento e o monitoramento por agentes de trânsito, mas, por outro lado, o aumento tecnológico e de fácil acesso à informações providas pelas placas automobilísticas abre múltiplas possibilidades para novos empreendimentos, com opções de sustentabilidade e qualidade de serviços. A proposta desta monografia é aplicar técnicas de Visão Computacional, em conjunto com Redes Neurais Convolucionais, com intuito de criar um sistema escrito na linguagem de programação Python que reconheça placas de identificação veiculares brasileiras (modelo anterior às placas padronizadas no Mercosul). O sistema ALPR desenvolvido apresentou resultados significativos utilizando o conjunto de dados UFPR-ALPR, sendo capaz de atingir um *recall* de 98,81% na detecção de placas veiculares, tendo melhor desempenho que os sistemas desenvolvidos por Laroca et al. (2018) e Laroca et al. (2019) (com *recall* de 98,33% e 98.67%, respectivamente). Além disso, apresentou precisão de 76,90% no reconhecimento de caracteres.

Palavras-chaves: ALPR, aprendizado de máquina, detecção de placas veiculares, reconhecimento de caracteres, redes neurais convolucionais, python e visão computacional.

Abstract

Automatic License Plate Recognition (ALPR) aims to detect and read the license plate text in either one or multiple vehicles in an image. ALPR is a frequent study field with several applications in everyday life, such as traffic surveillance and access control to parking lots. Considering the sudden increase in vehicle numbers and transportation systems in Brazil during the current century, the vehicle license plate has become similar to a personal identification, since companies in partnership with the government can access the person's data based on license plate of their vehicle. This significant increase in the number of vehicles makes the management and monitoring jobs harder. On the other hand, the technological advances and easy access to information provided by vehicle license plates open up new possibilities of new undertakings with sustainable option and better service quality. The purpose of this thesis is to apply Computer Vision techniques together with Convolutional Neural Networks in order to create a system written in Python programming language that recognizes Brazilian license plates (model prior to standard plates in Mercosur). The developed ALPR system presented meaningful results. Using the UFPR-ALPR dataset, it was capable of reaching a recall of 98,81% in license plate detection, performing better than the systems developed by Laroca et al. (2018) and Laroca et al. (2019) (with a recall of 98,33% and 98,67%, respectively). It was also possible to achieve a 76,90% accuracy in character recognition.

Keywords: ALPR, machine Learning, license plate detection, character recognition, convolutional neural networks, python and computer vision.

Lista de Ilustrações

Figura 1 - Modelo antigo das Placas de Identificação Veiculares
Figura 2 - Modelo novo de Placa de Identificação Veicular
Figura 3 - Cor dos caracteres alfanuméricos da PIV
Figura 4 - Fotografia de uma avenida com automóveis
Figura 5 - A aparência 2D dos objetos pode mudar radicalmente com o ponto de vista
Figura 6 - Placa em diferentes posições e focos de iluminação
Figura 7 - Etapas de processamento de imagens de acordo com Gonzalez e Woods (2007)
Figura 8 - Exemplo de uma rede neural convolucional e suas diferentes camadas
Figura 9 - Extração de característica - Detecção de bordas do claro para o escuro
Figura 10 - Exemplo da utilização de um padding
Figura 11 - Gráfico de linha de ativação linear retificada para entradas negativas e positivas
Figura 12 - Representação de pooling máximo e pooling médio
Figura 13 - Processamento de uma camada totalmente conectada
Figura 14 - Comparação do YOLOv4 com outros Detectores de Objeto de última Geração
Figura 15 - Arquitetura Tesseract OCR engine
Figura 16 - Exemplo de imagens do dataset UFPR-ALPR
Figura 17 - Processo de criação das anotações
Figura 18 - Anotação de uma imagem
Figura 19 - Imagem exemplo de caso que nenhuma placa foi detectada pela rede
Figura 20 - Imagem da placa recortada após ser reconhecida pelo modelo
Figura 21 - Exemplo do resultado da etapa 1 do Filtro 6
Figura 22 - Exemplo do resultado da etapa 2 do Filtro 6
Figura 23 - Exemplo do resultado da etapa 3 do Filtro 6
Figura 24 - Exemplo do resultado da etapa 4 do Filtro 6
Figura 25 - Exemplo do resultado da etapa 5 do Filtro 6
Figura 26 - Exemplo do resultado da etapa 6 do Filtro 6
Figura 27 - Imagem de um caractere segmentado de uma placa veicular

Lista de Tabelas

Tabela 1 - Arquitetura da versão Tiny YOLOv1
Tabela 2 - Arquitetura da versão Tiny YOLOv2
Tabela 3 - Arquitetura da versão Tiny YOLOv3
Tabela 4 - Arquitetura da versão Tiny YOLOv4
Tabela 5 - Arquitetura da YOLOv4 Tiny para detecção de placas veiculares
Tabela 6 - Resultados da detecção de placas veiculares
Tabela 7 - Resultados do reconhecimento de placas veiculares
Tabela 8 - Relação da quantidade de placas com a quantidade de caracteres reconhecidos
Tabela 9 - Resultados obtidos pelo sistema ALPR proposto comparado com o sistema de Laroca et al. (2018) e Laroca et al. (2019) utilizando o *dataset* UFPR ALPR

Lista de Abreviaturas e Siglas

ALPR - *Automatic License Plate Recognition*

AP - Average Precision (Precisão Média)

API - Application Programming Interface (Interface de Programação de Aplicação)

CET - Companhia de Engenharia de Tráfego

CNN - Convolutional Neural Network (Rede Convolucional)

COCO - Common Objects in Context (Objetos Comuns dentro de um Contexto)

CUDA - Compute Unified Device Architecture (Arquitetura de Dispositivo de Computação Unificada)

Denatran - Departamento Nacional de Trânsito

DETRAN - Departamento Estadual de Trânsito

FPS - Frames Per Second (Quadros Por Segundo)

Mercosul - Mercado Comum do Sul

OCR - Optical Character Recognition (Reconhecimento óptico de Caracteres)

OpenCV - Open Source Computer Vision Library (Biblioteca de Visão Computacional de Código Aberto)

PC - Personal Computer (Computador Pessoal)

PIV - Placa de Identificação Veicular

PNG - Portable Network Graphics

QR Code - Código de barras em formato quadrado

Refis - Registradores Fotográficos de Infrações em Semáforos

ReLu - Unidade Linear Retificada

RNAs - Redes Neurais Artificiais

Setran - Secretaria de Trânsito

YOLO - You Only Look Once (Você olha apenas uma vez)

2D - Bidimensional

3D - Tridimensional

Sumário

1 Introdução 11
1.1 Contexto 12
1.2 Justificativa 13
1.3 Objetivo 14
1.3.1 Objetivos Específicos 14
2 Revisão Bibliográfica 15
2.1 Placas de Identificação Veicular 15
2.2 Visão Computacional 17
2.3 Detecção de objetos 3D 20
2.4 Processamento de Imagens Digitais 21
2.4.1 OpenCV 23
2.5 Redes Neurais Convolucionais 25
2.5.1 Camada de Convolução 26
2.5.2 Camada de normalização em lote 28
2.5.3 Unidade Linear Retificada (ReLu) 28
2.5.4 Camada de Pooling 29
2.5.5 Camada totalmente conectada 30
2.5.6 Camada Soft-max 31
2.6 You Only Look Once (YOLO) 31
2.6.1 Versões anteriores 32
2.6.2 YOLOv4 35
2.7 Reconhecimento de Caracteres 38
2.7.1 Reconhecimento Óptico de Caracteres 39
2.7.2 Tesseract 39
2.8 Conjunto de dados UFPR-ALPR 42
2.9 Trabalhos Relacionados 43
3 Desenvolvimento 46
3.1 Proposta da Solução 46
3.2 Técnicas utilizadas 48
3.3 Conjunto de Dados 49
3.3.1 UFPR-ALPR Dataset 49
3.4 Materiais 49
3.5 Especificações técnicas 50
3.6 Desenvolvimento do Sistema 51
3.6.1 Criação do dataset ou anotando as imagens 51

3.6.2 Configurando e treinando a rede YOLOv4 Tiny 52
3.6.3 Convertendo o modelo para TensorFlow 55
3.6.4 Pré-processamento das imagens 56
3.6.5 Reconhecendo os caracteres da placa automobilística 62
3.6.6 Pós-processamento das imagens 63
4 Resultados e discussões 65
4.1 Métodos de Avaliação 65
4.2 Pré-Requisitos dos Testes 65
4.3 Execução dos Testes 66
4.4 Resultados 67
4.5 Análise e Discussões 70
5 Conclusões e Trabalhos Futuros 72
5.1 Sugestões para Trabalhos Futuros 72
Referências 74

1 Introdução

O aumento do desenvolvimento tecnológico na segunda metade do século XX levou à modernização de diversos sistemas e práticas do cotidiano, substituindo os antigos métodos analógicos e manuais por novos métodos tecnológicos e automáticos. A indústria automobilística não escapou dessa tendência, e, por consequência, foram criadas novas maneiras de integrar a placa de identificação veicular com a dinâmica das grandes cidades e com os centros tecnológicos.

Seguindo as ideias apresentadas por Moreira (2006), a velocidade do desenvolvimento tecnológico também trouxe a popularização de certas tecnologias. Afinal, como elas não eram mais consideradas de última geração, seu preço caía substancialmente, permitindo que até mesmo as classes mais baixas da população tivessem acesso a computadores e aparelhos de televisão.

Com essa popularização dos chamados PCs (*Personal Computer*), foram abertos novos ramos trabalhistas, como os de programadores e analistas de sistemas. Por consequência, também foram criadas bibliotecas de livre acesso, desenvolvidas por pessoas pertencentes à área, entusiastas de novas tecnologias ou até mesmo por grandes empresas (casos mais recentes). A visão computacional e o aprendizado de máquina são áreas atualmente populares e contam com um considerável número de bibliotecas.

O Brasil, como apontado por Moreira (2006), tem um histórico de adoção de novas tecnologias. Como mostrado em matéria jornalística veiculada no *site* G1 (2012), a primeira lombada eletrônica do país foi instalada no dia 20 de agosto de 1992, completando, em 2020, 29 anos de existência. Quando questionada pelo jornal, a Setran (Secretaria de Trânsito) afirmou que, apesar da falta de dados sobre o local, é consenso que, desde a instalação do equipamento, o índice de acidentes foi reduzido, uma vez que a visualização do equipamento desperta nos motoristas cautela e precaução, a fim de evitar uma multa.

Esse caso deixa evidente como a utilização de novas tecnologias pode melhorar o cotidiano da população. Logo, buscando a inovação em diversos pontos científicos e tecnológicos, o aprendizado de máquina e a visão computacional são integrados em *hardwares* mais populares, tornando triviais problemas que, antes, requeriam mão de obra humana.

Outro exemplo deste fenômeno é a operação de detecção e reconhecimento de placas de identificação veicular para fins comerciais e de segurança. Basta lembrar das cancelas de estacionamento que só abrem para veículos de placas registradas, tão comuns no cotidiano.

Neste caso, quando se trata de sistemas de reconhecimento automático de placas veiculares aplicados em cancelas de estacionamento, condomínios e outros diversos cenários, outra vertente que pode ser explorada é a sustentabilidade. Com efeito, de acordo com o estudo realizado por SANTA et al. (2018), o uso do sistema integrado digitalmente diminui os gastos de papel. Desta forma, donos de estabelecimento teriam uma redução mensal de seus gastos, podendo investir em outras áreas de seu empreendimento.

1.1 Contexto

Com os avanços tecnológicos das últimas décadas em inteligência artificial e aprendizado de máquina, pesquisas relacionadas ao reconhecimento automático de placas automobilísticas tornaram-se tendência. Nestas pesquisas, com a aplicação de algoritmos, o computador é capaz de aprender a detectar o objeto. Esses avanços, em conjunto com a disponibilidade de bibliotecas e com o retorno de algoritmos (possibilidades antes inimagináveis, dada a falta de capacidade tecnológica dos equipamentos produzidos entre a década de 50 e inicio da década de 90) permitiram, ainda que parcialmente, replicar as funcionalidades do olho humano, como detecção e reconhecimento de objetos e caracteres. Essa réplica pode ser aplicada na detecção e reconhecimento automático de placas automobilísticas — traduzido do inglês *Automatic License Plate Recognition* (ALPR) —, sendo passível de integração em áreas com fins comerciais, de segurança e sustentabilidade.

Esse tipo de *software* é desenvolvido com detectores de objetos que podem ser construídos a partir de diversos tipos de algoritmos. No Brasil, seu uso mais comum está relacionado a radares de velocidade, mas também tem apresentado participação crescente em portarias de condomínios e prédios: o banco de dados guarda a identificação das placas veiculares dos moradores e, depois, o sistema pode compará-las com a placa do carro que tem intenção de entrar no condomínio.

1.2 Justificativa

A utilização de tecnologias como detecção e reconhecimento de placas automotivas apresenta um aspecto comercial, de segurança e sustentabilidade. Essa tecnologia pode ser comercializada para uso em diversos setores, melhorando alguns aspectos de problemas já existentes. Atualmente é possível testemunhar esse tipo de sistema em uso em algumas situações do cotidiano, como cancelas de estacionamentos privados, para gerenciar a entrada e saída de carros registrados.

Também é possível observar um aspecto de segurança, e o maior exemplo disponível é a atuação do Departamento de Trânsito Brasileiro (DETRAN). De acordo com a matéria referente ao aumento de multas de trânsito na cidade de São Paulo realizada por Castro (2019) para o jornal Folha de São Paulo, em 6 de dezembro de 2019, de um total de 6.693.510 multas relacionadas a transitar em local e área fora do horário permitidos, 1.548.858 foram captadas de maneira digital, através de sistemas de detecção e reconhecimento automáticos de placas. Pode-se considerar também a supervisão feita pelo DETRAN através de câmeras instaladas em rodovias e radares, que são acompanhados de um pequeno computador com a capacidade de identificar os carros que estão circulando pela rodovia.

Um terceiro aspecto a ser considerado é a sustentabilidade. Esse aspecto surge como consequência da digitalização de recursos que antes eram impressos, a exemplo dos *tickets* de estacionamento. Estudos como o de SANTA et al. (2018) mostram que, com a diminuição do gasto de papel, o usuário de um estacionamento não precisaria de um *ticket* relativo ao seu carro e horário de entrada, uma vez que essas informações estariam atreladas no sistema diretamente com a placa de seu carro. O cliente teria apenas que informar qual a identificação de seu veículo para efetuar um pagamento da conta referente à sua estadia. Essa mudança traria benefícios para o dono do local, que economizaria com os gastos relacionados ao consumo de papel (compra de papel, impressão de *tickets* e manutenção da impressora). Ele poderia realocar o orçamento destinado a esses itens para outras utilidades e possíveis melhorias de seu empreendimento.

1.3 Objetivo

Criar um sistema baseado em redes convolucionais que detecte e reconheça, de forma automática, placas de automóveis brasileiros (modelos anteriores àquele padronizado pelo Mercosul). Uma vez que a placa do automóvel é detectada, passa-se à obtenção e extração do código identificador do veículo, por meio da aplicação de técnicas da visão computacional e de algoritmos de reconhecimento de caracteres.

1.3.1 Objetivos Específicos

- Aprender a usar técnicas para detecção de placas automotivas utilizando ferramentas de desenvolvimento na área de visão computacional e possíveis redes neurais a serem aplicadas;
- Implementar um sistema em linguagem de programação Python que altere e melhore as imagens para detecção, também conhecido como processamento de imagem;
- Comparar os resultados obtidos com os resultados de diversos trabalhos relacionados;
- Aplicar técnicas de reconhecimento de caracteres.

2 Revisão Bibliográfica

Neste capitulo, são apresentados os conceitos teóricos necessários para a realização dos objetivos propostos na monografia.

2.1 Placas de Identificação Veicular

As placas de identificação de veículos no Brasil são emitidas pelos Departamentos Estaduais de Trânsito de cada estado e do Distrito Federal, seguindo um sistema alfanumérico comum em todo o país.

Figura 1 - Modelo antigo das Placas de Identificação Veiculares

Fonte: Wikipédia (2010)

De acordo com Brasil (2019), o novo modelo de Placas de Identificação Veicular (PIV), que segue o padrão estabelecido pelo Mercosul (Mercado Comum do Sul), é exigido desde de 31 de janeiro de 2020 nos seguintes casos:

- Registro de veículo 0 km;
- Mudança de categoria do veículo;
- Em caso de furto, extravio, roubo ou danos na placa (inclusive dano à tarjeta e rompimento do lacre da placa traseira padrão cinza);
- Mudança de município ou de Estado;

- Quando o veículo for reprovado em vistoria veicular nos procedimentos de transferência com observações sobre a placa e/ou lacre (exemplo: placa não refletiva);
- Necessidade de instalação de placa adicional traseira.

Em conformidade com DETRAN (201-), a PIV deve ser revestida, em seu anverso, de película retrorrefletiva, na cor branca com uma faixa na cor azul na margem superior, contendo ao lado esquerdo o logotipo do MERCOSUL, ao lado direito a bandeira do Brasil e ao centro o nome BRASIL.

O padrão de estampagem é composto de 07(Sete) caracteres alfanuméricos, em alto relevo, na sequência LLLNLNN, com igual espaçamento e combinação aleatória, distribuída e controlada pelo Denatran (Departamento Nacional de Trânsito). Além disso, o novo modelo de PIV, ilustrada na Figura 2, não possuirá lacre. O QR Code (Código de barras em formato quadrado) irá substituí-lo. O caractere "L" refere-se à letra, e o caractere "N" refere-se ao numeral.

Figura 2 - Modelo novo de Placa de Identificação Veicular

Fonte: DETRAN (201-)

Dessa forma, os novos modelos de placas e suas respectivas cores são ilustrados na Figura 3.

Figura 3 - Cor dos caracteres alfanuméricos da PIV

Fonte: DETRAN (201-)

2.2 Visão Computacional

Em conformidade com Forsyth e Ponce (2002), o humano tem facilidade em perceber a estrutura tridimensional (3D) do mundo, como por exemplo, a percepção da visão tridimensional de automóveis em uma avenida. O sistema visual humano não tem problemas para interpretar elementos que podem aparecer em uma imagem. Tais elementos incluem a forma, o tamanho a placa, a cor da placa e o sombreamento de cada automóvel na avenida. Tome como exemplo uma fotografia ilustrada na Figura 4, qualquer ser humano poderia facilmente perceber os elementos desta imagem.

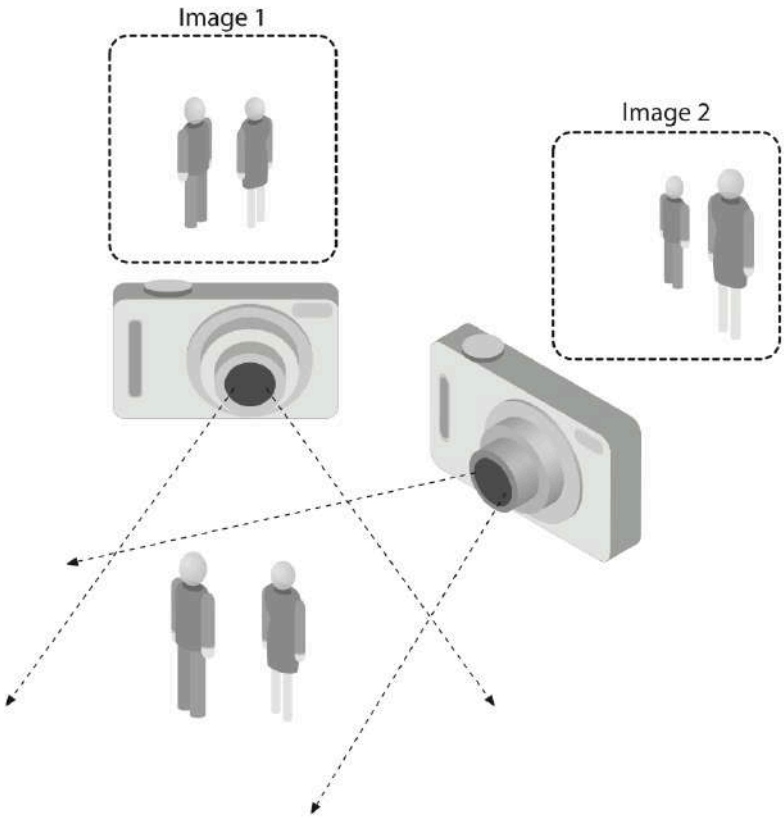
Figura 4 - Fotografia de uma avenida com automóveis

Fonte: UFPR-ALPR... (2018)

Os algoritmos de visão computacional, por vez, necessitam de um conjunto de imagens para reconstruir as suas propriedades, como forma, iluminação e distribuição de cores. Este processo de reconstrução são propensos à erros, devido à capacidade computacional, falta de qualidade da imagem alvo e à escolha do algoritmos para realizar o processamento.

Segundo Bradski e Kaehler (2008), dada uma visão bidimensional (2D) de um mundo tridimensional, não há uma maneira única de reconstruir o sinal 3D. Formalmente, esse problema colocado não tem solução única ou definitiva, pois a mesma imagem 2D pode representar inúmeras combinações de cenas 3D.

Figura 5 - A aparência 2D dos objetos pode mudar radicalmente com o ponto de vista



Fonte: Bradski e Kaehler (2008)

Conforme Gad e John (2018), atualmente, uma aplicação muito desafiadora na área de Visão Computacional é o reconhecimento de objetos. O reconhecimento pode basear-se em diferentes dados, como áudio, imagem e texto. Para imagem, o reconhecimento é uma maneira eficiente devido à abundância de informações que podem ser úteis na tarefa.

Existem diversos objetos no mundo, e diferenciá-los é uma tarefa complexa para Visão Computacional. Objetos diferentes podem ter aparência visual semelhante, exceto por detalhes sutis. Além disso, o mesmo objeto aparece diferentemente com base no ambiente. Por exemplo, com base na luz, ângulo de visão, distorção e oclusão, o mesmo objeto aparece de maneira diferente, como demonstrado na Figura 6.

Figura 6 - Placa em diferentes posições e focos de iluminação

Fonte: Própria

2.3 Detecção de objetos 3D

A detecção de objetos 3D pode ser definido como a percepção de um objeto no mundo real (como exemplo, Placas de Identificação Veicular) através de uma imagem digital ou vídeo.

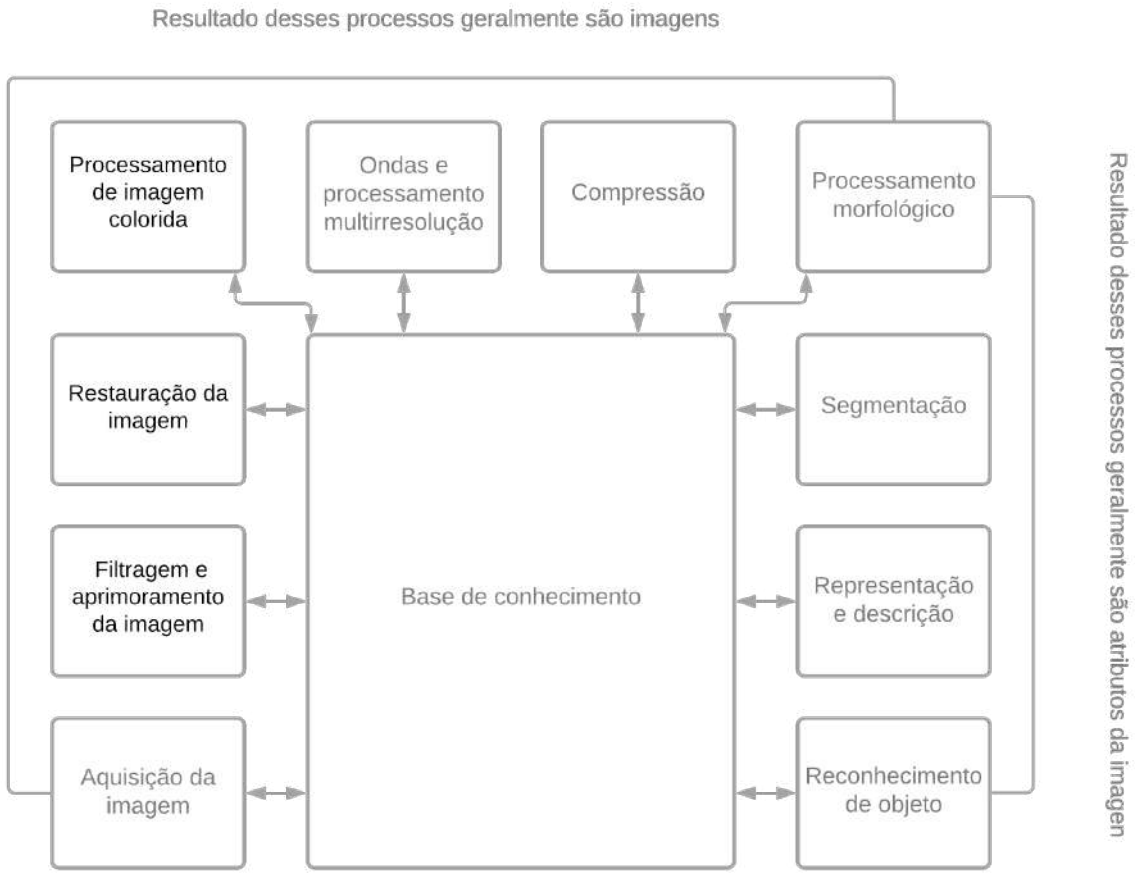
Este é um tema recorrente de pesquisa na área de Visão Computacional, devidos à sua alta complexidade e diferentes variáveis que podem alterar a precisão e técnicas dos algoritmos utilizados.

Mohamad (2013) diz que a detecção de objetos é dividido de duas maneiras: por instância e por categoria. A detecção por instância se baseia em identificar um objeto específico em uma imagem ou vídeo, por exemplo, identificar o modelo de um carro. Enquanto a detecção por categorização detecta um item dentro de uma categoria, por exemplo, detectar que a imagem ou vídeo contém um carro qualquer, independente do modelo.

2.4 Processamento de Imagens Digitais

Em concordância com Gonzalez e Woods (2007), uma imagem é definida como uma função de duas dimensões, f(x, y), tal que x e y são coordenadas espaciais e a amplitude de f em certa coordenada (x, y) pode ser considerada a intensidade ou nível de cinza da imagem nesse ponto em específico. Quando x, y, e os valores de intensidade de f são quantidades finitas e discretas ou seja somente valores inteiros, a imagem é chamada de digital.

Figura 7 - Etapas de processamento de imagens de acordo com Gonzalez e Woods (2007)



Fonte: Gonzalez e Woods (2007)

Em concordância com Gonzalez e Woods (2007), uma breve descrição para cada passo do processamento de imagem seria:

- **Aquisição da imagem:** é o processo de carregamento da imagem;
- **Filtragem e aprimoramento da imagem:** é o processo de manipulação da imagem que resulta em uma imagem mais adequada para uma aplicação específica;
- **Restauração da imagem:** é uma área que também lida com melhorias da aparência da imagem, entretanto, ao contrário do aprimoramento, que é subjetivo, a restauração da imagem é objetiva, no sentido de que as técnicas de restauração tendem a se basear em modelos matemáticos ou probabilísticos de degradação da imagem;
- **Processamento da imagem colorida:** processo no qual as cores da imagem são processadas;
- **Ondas e processamento multirresolução:** baseia-se em representar imagens em vários graus de resolução;
- **Compressão:** lida com técnicas para redução do armazenamento necessário para salvar uma imagem, ou largura de banda necessária para transmiti-la;
- **Processamento morfológico:** lida com ferramentas para extrair componentes nas imagens que são úteis na representação e descrição do formato;
- **Segmentação:** é um procedimento que particiona a imagem em partes menores;
- **Representação e descrição:** é o processo que quase sempre segue a saída de um estágio de segmentação, no qual são dados de pixels brutos, constituindo o limite de uma região ou todos os pontos na própria região;
- **Reconhecimento de objeto:** é o processo que atribui um rótulo ao objeto da imagem, por exemplo “placa”.

Já Russ (1990), diz que o processamento de imagem, inclui métodos que começam com uma imagem (uma matriz de pixels, cada um com valor de brilho ou “escala de cinza” ou talvez com informações de cores) e terminam com uma imagem. Normalmente, a imagem resultante é de tamanho semelhante (número de pixels e número de níveis de cinza). O brilho dos pixels resultantes será, na maioria dos casos, modificado usando operações que levam em conta o valor original do pixel e de seus vizinhos, ou talvez sua posição ou relação espacial com muitos outros pixels.

Existem vários tipos de operações, todas elas pretendem produzir uma imagem modificada ou processada que enfatize algum aspecto da imagem original. Dessa forma, essas operações são consideradas técnicas de filtragem que selecionam certos tipos de dados de imagem. Na maioria das vezes, os recursos desejados são objetos, arestas, limites ou alguma outra estrutura definida. Dependendo do que é considerado de interesse, muitas vezes é possível construir uma operação de processamento adequada para extraí-la das informações de fundo (consideradas desinteressantes) na imagem, para que possam ser medidas. Às vezes, é necessário operar várias vezes na mesma imagem original, usando diferentes operações de processamento, para isolar diferentes tipos de informações.

2.4.1 OpenCV

Segundo a documentação oficial OpenCV (2020) e Bradski e Kaehler (2008), o OpenCV (*Open Source Computer Vision Library)* é uma biblioteca de código aberto que inclui centenas de algoritmos de visão computacional. Ele é construído com uma estrutura modular, o que indica que ele inclui diversas bibliotecas estáticas ou compartilhadas. A biblioteca foi escrita em linguagem de programação C e C++ e roda nos sistemas operacionais Linux, macOS e Windows. Além disso, existem desenvolvimentos de bibliotecas para Python, Ruby, Matlab e outras linguagens. O OpenCV foi desenvolvido com a eficiência computacional como grande foco, podendo ser utilizado em aplicações de tempo real.

Dentre todas as operações contidas no OpenCV (2020), é possível relacionar algumas delas com as etapas de processamento de imagens. Como por exemplo:

Operações para trocar ou realçar as cores presentes nas imagens, também conhecidas como trocas de ambientes de cores.

- **Trocas de ambientes de cores**: essa operação tem o objetivo de alterar a coloração de uma imagem. Considerada um dos métodos mais importantes, essa operação transforma uma imagem com três canais de cores (RGB) para dois canais de cores (escala de cinzas);

Operações que fazem transformações geométricas nas imagens:

- **Redimensionamento**: métodos para alterar a escala da imagem, ou seja, é possível alterar a imagem sem causar distorções;

Operações para remover ruídos das imagens, conhecidas como desfoque de imagem ou suavização de imagem.

- **Desfoque Gaussiano:** eficaz na remoção de ruído gaussiano de uma imagem.
- **Desfoque Médio:** eficaz contra ruído de sal e pimenta, também conhecido como ruído de impulso, de uma imagem.

Operações que limitam imagens em tons de cinza.

- **Limitação simples:** na qual para cada pixel, o mesmo valor de limite é aplicado. Se o valor do pixel for menor que o limite, ele é definido como 0, caso contrário, ele é definido como um valor máximo.
- **Limitação adaptativa:** na qual é utilizada para caso uma imagem tenha diferentes condições de iluminação em diferentes áreas. Seu algoritmo determina o limite de um pixel com base em uma pequena região ao seu redor. Portanto, são obtidos limites diferentes para regiões diferentes da mesma imagem, podendo resultar em melhores imagens com iluminação variável.

Operações que fazem transformações morfológicas, nas quais são algumas operações simples baseadas na forma da imagem. Normalmente é executado em imagens binárias (imagens em preto e branco).

- **Erosão:** funciona como a erosão do solo, desgastando a região branca na imagem ou o tamanho do objeto em primeiro plano diminui.
- **Dilatação:** esta operação faz exatamente o oposto da erosão, onde ela aumenta a região branca na imagem ou o tamanho do objeto em primeiro plano aumenta.

E por fim, inclui-se uma operação que **procura contornos em uma imagem binária** utilizando dois algoritmos criados por Suzuki e Abe (1985). Segundo a dupla, o primeiro determina as relações circundantes entre as bordas de uma imagem binária. Uma vez que as bordas externas e as bordas dos buracos têm uma correspondência um-para-um entre os componentes conectados de l-pixels e os buracos, respectivamente, o algoritmo proposto produz uma representação de uma imagem binária, da qual se pode extrair algum tipo de recursos sem reconstruir a imagem. O segundo algoritmo, que é uma versão modificada do primeiro, segue apenas as bordas externas, as quais são bordas que não são circundadas por buracos.

2.5 Redes Neurais Convolucionais

Conforme LeCun et al. (1998), Rede Neural Convolucional (*Convolutional Neural Network* - CNN) é um modelo de rede neural que incorpora as seguintes idéias: campos receptivos, compartilhamento de peso e subamostragem. É um tipo especial de Perceptron multicamadas treinado no modo supervisionado usando o algoritmo de retropropagação. CNN é capaz de realizar resultados de última geração em tarefas como reconhecimento de caracteres, reconhecimento de objetos, detecção facial, reconhecimento de fala, detecção de placas automotivas, pré-processamento de imagens e tarefas de segmentação.

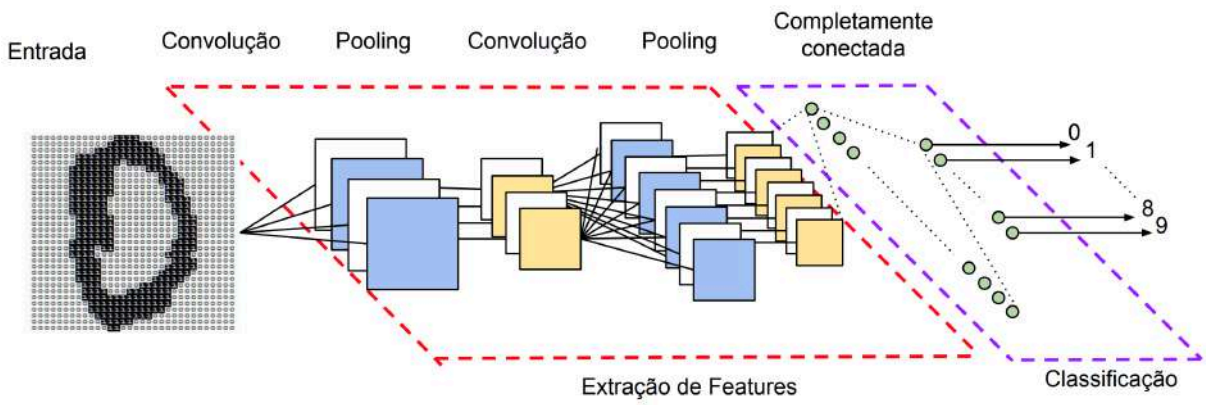
Uma CNN aprende recursos complexos e de alta dimensão a partir de um grande número de exemplos, o que a torna uma candidata para tarefas de reconhecimento de padrões. Redes convolucionais combinam três idéias arquitetônicas para garantir algum grau de mudança, escala e invariância de distorção: campos receptivos locais, pesos compartilhados (ou replicação de pesos) e subamostragem espacial ou temporal. Ao contrário das tarefas convencionais de reconhecimento de padrões, um dos benefícios da CNN está nos próprios recursos de extração e nas imagens que usa como entrada para o treinamento e teste da rede. A CNN cria recursos complexos a partir de uma grande coleção de exemplos e a complexidade dos recursos de aprendizagem com mais camadas. Isso é feito convolvendo sucessivamente a imagem de entrada com filtros para construir uma hierarquia de mapas de características. A hierarquia resulta em recursos e

aprendizado complexos, bem como invariante translacional e de distorção. Toda a CNN pode ser expressa como uma função de pontuação, em que os pixels da imagem bruta são fornecidos como entrada e determinam a categoria ou a pontuação da classe no final.

Segundo Fujita et al. (2018), CNN consiste em várias camadas de rede, como camada de Convolução, camada de Normalização em lote, função de ativação ReLu (*Rectified Linear Unit* - unidade linear retificada), camada Pooling, e etc. Dependendo do tamanho da entrada, o número de camadas pode ser aumentado ou diminuído. Não é necessário usar todas as camadas da rede. A vantagem da CNN é que ela não precisa de nenhum pré-processamento ou extração manual de recursos por outros métodos. Em geral, os recursos são extraídos de maneira hierárquica mapeando pixels brutos da imagem de entrada e, além disso, são classificados usando camadas totalmente conectadas. Os parâmetros de rede são ajustados de forma inteligente para obter o mais alto desempenho de classificação.

Sendo assim, na Figura 8 é ilustrado uma CNN e em seguida é apresentado basicamente a estrutura de uma CNN.

Figura 8 - Exemplo de uma rede neural convolucional e suas diferentes camadas



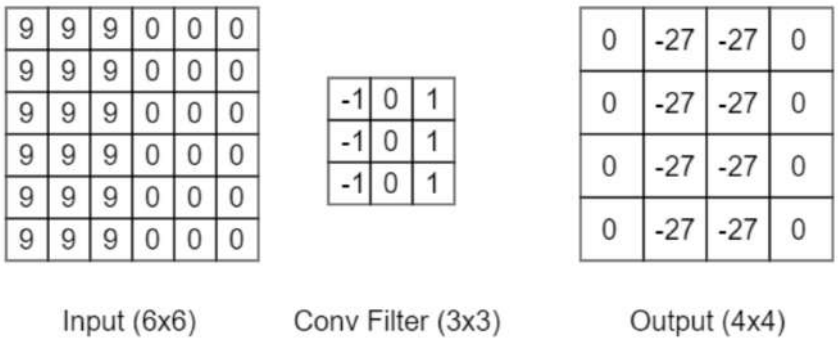
Fonte: Vargas (2016)

2.5.1 Camada de Convolução

Segundo LeCun, Bengio e Hinton (2015), a camada de convolução é representada por neurônios conectados a sub-regiões a partir das imagens de entrada ou saída da camada anterior, na qual é responsável por aplicar filtros

convolucionais que varrem toda a imagem e extraem mapas de recursos. Com objetivo dos filtros aprenderem diferentes tipos de características da entrada, cada filtro convolucional contém um conjunto de pesos. A Figura 9 abaixo ilustra uma detecção de borda vertical de uma entrada 6x6, com valores em escala de cinza, na qual um filtro de tamanho 3x3 à varre a cada um pixel por um, ou seja, com um *stride* de 1, sendo o *stride* um componente vital para a construção dessa camada, indicando de que forma o filtro avança sobre a entrada.

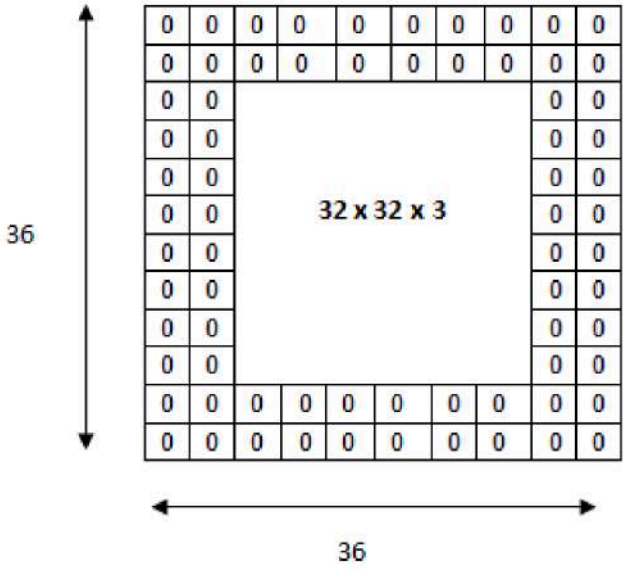
Figura 9 - Extração de característica - Detecção de bordas do claro para o escuro



Fonte: Varty (2018)

A camada convolucional possui outra técnica que também pode ser aplicada com intuito de obter características contidas próximo das bordas, o *padding*. A Figura 10 abaixo ilustra essa técnica, na qual basicamente, insere zeros no contorno das bordas da imagem.

Figura 10 - Exemplo da utilização de um padding



Fonte: Wei et al. (2020)

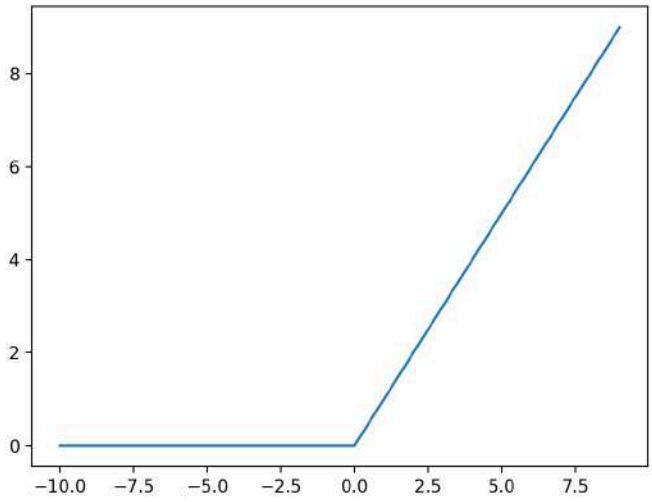
2.5.2 Camada de normalização em lote

Segundo seus criadores Ioffe e Szegedy (2015), a camada de normalização ocorre para cada mini-lote de treinamento, permitindo usar taxas de aprendizado muito mais altas e atingindo a mesma precisão com 14 vezes menos etapas de treinamento necessários. Sendo capaz de superar o modelo original sem o uso da normalização. Além de exercer sua função em normalizar os dados fornecidos à cada camada oculta.

2.5.3 Unidade Linear Retificada (ReLu)

ReLu é uma camada de ativação não linear que faz parte das camadas convolucional e normalização em lote, na qual para cada elemento da entrada, a operação de limitação (*thresholding*) é realizada, onde valores menores que zero são definidos como zero como ilustrado na Figura 8.

Figura 11 - Gráfico de linha de ativação linear retificada para entradas negativas e positivas



Fonte: Brownlee (2019)

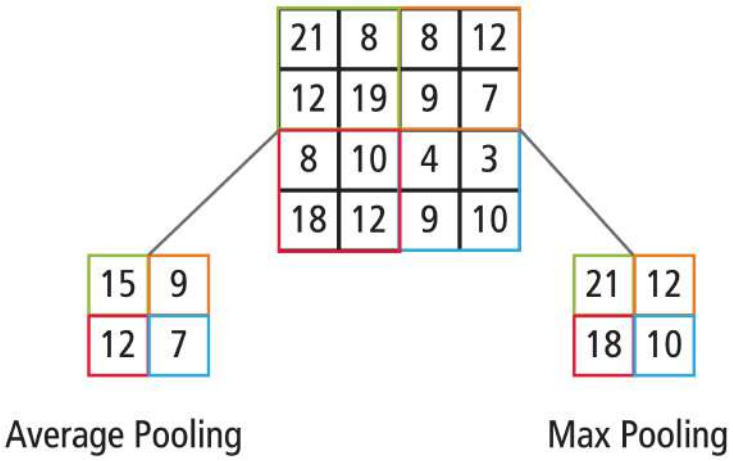
De acordo com Krizhevsky et al. (2012), a função de ativação produz tempo e erro de treinamento menor, comparado com as funções sigmoide e tangente hiperbólica.

2.5.4 Camada de Pooling

De acordo com Hijazi (2015), a camada de pooling ou também conhecida como subamostragem reduz a resolução dos recursos de entrada, tornando os recursos robustos contra ruído e distorção, ou seja, ela e reduz a dimensionalidade de um mapa de característica fornecido e produz outro mapa de característica.

Existem duas formas principais para criação desse novo mapa de característica, selecionando o valor maior (*pooling* máximo) ou fazendo a média dos valores (*pooling* médio). Na Figura 12 é ilustrado como é feita a criação do mapa de caraterística utilizando *pooling* máximo ou médio.

Figura 12 - Representação de pooling máximo e pooling médio

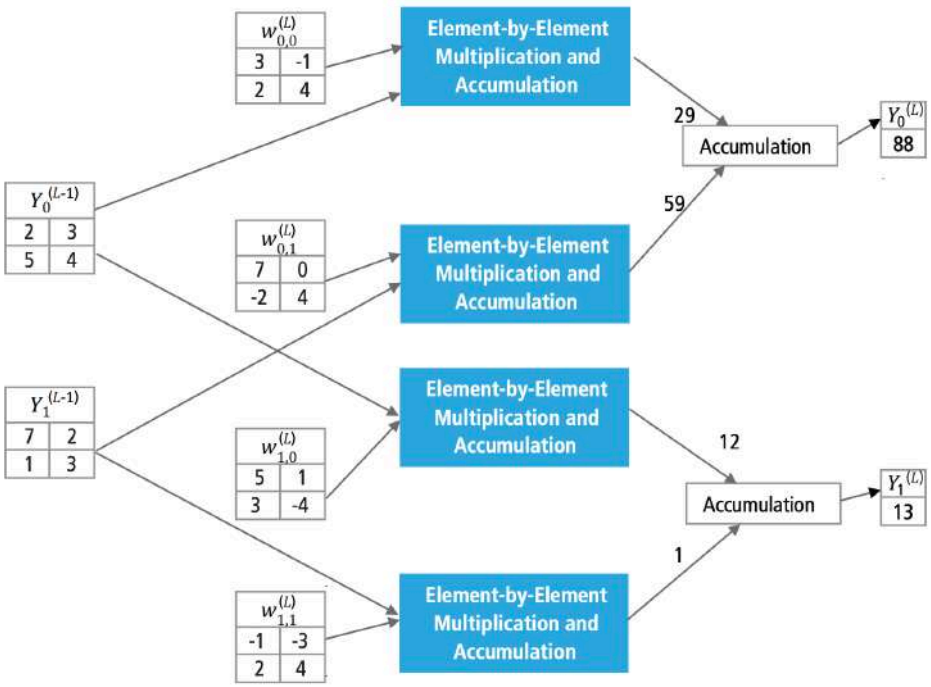


Fonte: Hijazi (2015)

2.5.5 Camada totalmente conectada

Em concordância com Hijazi (2015), as camadas totalmente conectadas são frequentemente usadas como camadas finais de uma CNN. Essas camadas somam matematicamente uma ponderação da camada anterior de recursos, indicando a mistura precisa de “ingredientes” para determinar um resultado de saída de destino específico. No caso de uma camada totalmente conectada, todos os elementos de todos os recursos da camada anterior são usados no cálculo de cada elemento de cada recurso de saída. A Figura 13 explica a camada L totalmente conectada. A Camada L-1 tem dois recursos, cada um dos quais é 2x2, ou seja, tem quatro elementos. A camada L tem dois recursos, cada um com um único elemento.

Figura 13 - Processamento de uma camada totalmente conectada



Fonte: Hijazi (2015)

2.5.6 Camada Soft-max

Esta camada aplica uma função *soft-max* à amostra de dados de entrada. Matematicamente, é uma função exponencial normalizada, que comprime as amostras de dados multidimensionais no intervalo de 0 a 1. Ela ajuda a reduzir os valores discrepantes sem remover da amostra de dados. Além disso, a camada de classificação contém a função de perda, rótulo de saída e tamanho.

2.6 You Only Look Once (YOLO)

Segundo Redmon (2015), YOLO (*You Only Look Once*) é um detector de objetos baseado na linguagem de programação C e CUDA, ambos herdados do framework que é utilizado para sua criação, o Darknet. O YOLO reformula o problema de detecção de objetos em um único problema de regressão que consiste em prever *bounding boxes* e probabilidades de classe diretamente dos pixels da imagem. Logo, quando uma imagem é submetida ao sistema, ela é dividida em uma

grade S × S de células quadradas. Cada objeto está localizado dentro de uma célula, e cada célula é responsável por detectar os objetos que ela contém. Isso é realizado por uma rede neural que prevê *bounding boxes* juntamente com as probabilidades previstas, chamadas de confiança, representadas pelo símbolo *S*, podem ser definidas pela fórmula:

$$\mathcal{S} = P(Obj) \cdot IOU_{pred}^{truth},$$

Onde *P(Obj)* é a probabilidade de existir um objeto dentro da caixa e *IOU* mede a intersecção dentre a caixa predita e qualquer caixa reconhecida como verdadeira, dado o treinamento. Ele também computa simultaneamente *C* probabilidades de classes condicionais $P(C_i|Obj)$, $1 \leq i \leq C$, onde cada C_i é uma classe do problema.

Em concordância com Redmon e Farhadi (2016), detectores anteriores ao YOLO, como R-CNN e Fast-CNN utilizavam um método diferente, no qual se baseavam em classificadores para realizar a detecção, além de exigirem milhares de avaliações de rede para fazerem previsões, ao contrário da YOLO, que faz com uma única avaliação de rede.

2.6.1 Versões anteriores

Em sua primeira versão, de acordo com Redmon (2015), a previsão da rede foi capaz de compreender as coordenadas e os tamanhos das *bounding boxes* (coordenada no eixo vertical, coordenado no eixo horizontal, largura e altura) juntamente com a confiança e as probabilidades da classe *C*. Sua arquitetura foi baseada no modelo GoogLeNet, com 24 camadas convolucionais seguidas por 2 camadas totalmente conectadas. A função de perda aplicada no treinamento foi baseada no erro da soma quadrática.

Tabela 1 - Arquitetura da versão YOLOv1 Tiny

Layer	***Type***	***Filters***	***Size/Stride***	***Input***	***Output***
0	conv	16	3x3/1	448x448x3	448x448x16
1	max		2x2/2	448x448x16	224x224x16
2	conv	32	3x3/1	224x224x16	224x224x32
3	max		2x2/2	224x224x32	112x112x32
4	conv	64	3x3/1	112x112x32	112x112x64
5	max		2x2/2	112x112x64	56x56x64
6	conv	128	3x3/1	56x56x64	56x56x128
7	max		2x2/2	56x56x128	28x28x128
8	conv	256	3x3/1	28x28x128	28x28x256
9	max		2x2/2	28x28x256	14x14x256
10	conv	512	3x3/1	14x14x256	14x14x512
11	max		2x2/2	14x14x512	7x7x512
12	conv	1024	3x3/1	7x7x512	7x7x1024
13	conv	256	3x3/1	7x7x1024	7x7x256
14	detection				

Fonte: Darknet

Uma segunda versão YOLO, a YOLOv2, foi lançada com objetivo de aumentar a velocidade, reduzir erros de localização e melhorar a métrica de avaliação Recall conforme Redmon e Farhadi (2016). A normalização em lote e a adoção de caixas de âncora (em vez de prever diretamente as coordenadas) levaram a uma melhor recuperação e a menos erros de localização no YOLOv2 em comparação com o YOLOv1. Além disso, Darknet-19 foi usado como um novo modelo de classificação, com 19 camadas convolucionais e seis camadas de agrupamento máximo.

Tabela 2 - Arquitetura da versão YOLOv2 Tiny

Layer	***Type***	***Filters***	***Size/Stride***	***Input***	***Output***
0	conv	16	3x3/1	416x416x3	416x416x16
1	max		2x2/2	416x416x16	208x208x16
2	conv	32	3x3/1	208x208x16	208x208x32
3	max		2x2/2	208x208x32	104x104x32
4	conv	64	3x3/1	104x104x32	104x104x64
5	max		2x2/2	104x104x64	52x52x64
6	conv	128	3x3/1	52x52x64	52x52x128
7	max		2x2/2	52x52x128	26x26x128
8	conv	256	3x3/1	26x26x128	26x26x256
9	max		2x2/2	26x26x256	13x13x256
10	conv	512	3x3/1	13x13x256	13x13x512
11	max		2x2/1	13x13x512	13x13x512
12	conv	1024	3x3/1	13x13x512	13x13x1024
13	conv	512	3x3/1	13x13x1024	13x13x512
14	conv	425	1x1/1	13x13x512	13x13x425
15	detection				

Fonte: Darknet

A segunda versão mais recente lançada por Redmon e Farhadi (2018), prevê caixas delimitadoras em três escalas diferentes, apresentando melhor desempenho em imagens pequenas. Além de introduzir um novo modelo de classificação com 53 camadas convolucionais, nomeada como Darknet-53. Com essas melhorias, é capaz de mostrar resultados melhores e mais rápidos do que a ResNet-10, e teve precisão semelhante ao da ResNet-152, mas com o dobro da velocidade.

Tabela 3 - Arquitetura da versão YOLOv3 Tiny

Layer	***Type***	***Filters***	***Size/Stride***	***Input***	***Output***
0	conv	16	3x3/1	416x416x3	416x416x16
1	max		2x2/2	416x416x16	208x208x16
2	conv	32	3x3/1	208x208x16	208x208x32
3	max		2x2/2	208x208x32	104x104x32
4	conv	64	3x3/1	104x104x32	104x104x64
5	max		2x2/2	104x104x64	52x52x64
6	conv	128	3x3/1	52x52x64	52x52x128
7	max		2x2/2	52x52x128	26x26x128
8	conv	256	3x3/1	26x26x128	26x26x256
9	max		2x2/2	26x26x256	13x13x256
10	conv	512	3x3/1	13x13x256	13x13x512
11	max		2x2/1	13x13x512	13x13x512
12	conv	1024	3x3/1	13x13x512	13x13x1024
13	conv	256	1x1/1	13x13x1024	13x13x256
14	conv	512	3x3/1	13x13x256	13x13x512
15	conv	255	1x1/1	13x13x512	13x13x255
16	yolo				
17	route 13				
18	conv	128	1x1/1	13x13x256	13x13x128
19	up-sample		2x2/1	13x13x128	26x26x128
20	route 19, 8				
21	conv	256	3x3/1	13x13x284	13x13x256
22	conv	256	1x1/1	13x13x256	13x13x256
23	yolo				
24	detection				

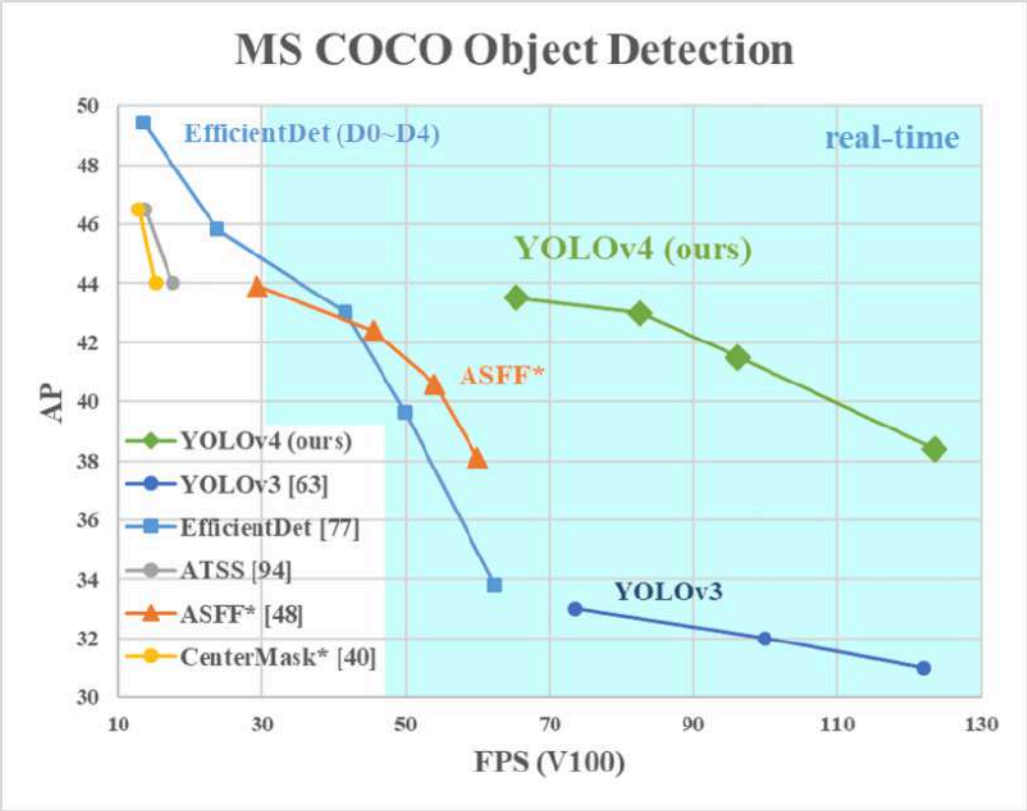
Fonte: Darknet

2.6.2 YOLOv4

Segundo Bochkovskiy et al. (2020), a YOLOv4, mais nova iteração do YOLO continua seguindo o modelo base de simplicidade e acessibilidade para um detector em tempo real, além de melhorar o desempenho do YOLO no geral. Como mostrado

na Figura 14 abaixo, o YOLOv4 treinado com o *dataset* COCO (*Common Objects in Context*), um conjunto de dados de objetos gerais apresentados em diversos contextos, sem se especificar em nenhum tipo de objeto a ser identificado, e sendo treinado com apenas uma única GPU, apresenta desempenho similar a grandes detectores de objetos como o EfficientDet, um detector implementado com Keras e Tensorflow, e o ASFF, um detector de objetos de disparo único implementado com YOLOv3 e MobileNetV2. Ele apresenta um melhor desempenho nos dois vértices do gráfico quando comparado somente com o EfficientDet, mostrando um aumento de 12% de desempenho em FPS (*Frames Per Second*) e de 10% em AP (*Average Precision* - precisão média).

Figura 14 - Comparação da YOLOv4 com outros Detectores de Objeto de última geração



Fonte: Bochkovskiy et al. (2020)

De acordo com Bochkovskiy et al. (2020), a maioria dos detectores de objetos baseados em redes CNN servem apenas para sistemas de recomendação, deixando que outros sistemas sejam interpretados por redes com precisão alta, mas

desempenho lento. Melhorar a acurácia de detectores de objetos em tempo real permite sua utilização em sistemas diferentes do sistema de recomendação, agindo como um processo de gestão *stand-alone* (processo independente e individual) e de redução de interação humana.

A operação de detectores de objetos em tempo real em GPUs (*Graphics Processing Units*) permite sua utilização em massa devido a preços acessíveis, permitindo que um entusiasta de tecnologia utilize uma rede convolucional para tentar resolver seu problema, independente de grandes empresas com acesso a equipamentos de ultima geração.

As mais precisas redes neurais modernas não operam em tempo real e requerem um grande número de GPUs para treinamento com grandes tamanhos de *mini-batchs* (mini-lotes). Isso foi resolvido através do desenvolvimento de uma CNN, a YOLOv4, que opera em tempo real considerando uma GPU convencional, e esse treinamento necessita apenas de uma GPU comum.

Tabela 4 - Arquitetura da versão YOLOv4 Tiny

Layer	***Type***	***Filters***	***Size/Stride***	***Input***	***Output***
0	conv	32	3x3/2	416x416x3	208x208x32
1	conv	64	3x3/2	208x208x32	104x104x64
2	conv	64	3x3/1	104x104x64	104x104x64
3	route 2			1/2	104x104x32
4	conv	32	3x3/1	104x104x32	104x104x32
5	conv	32	3x3/1	104x104x32	104x104x32
6	route 5, 4				104x104x64
7	conv	64	1x1/1	104x104x64	104x104x64
8	route 2, 7				104x104x128
9	max		2x2/2	104x104x128	52x52x128
10	conv	128	3x3/1	52x52x128	52x52x128
11	route 10			1/2	52x52x64
12	conv	64	3x3/1	52x52x64	52x52x64
13	conv	64	3x3/1	52x52x64	52x52x64
14	route 13, 12				52x52x128
15	conv	128	1x1/1	52x52x128	52x52x128
16	route 10, 15				52x52x256

17	max		2x2/2	52x52x128	26x26x256
18	conv	256	3x3/1	26x26x256	26x26x256
19	route 18			1/2	26x26x128
20	conv	128	3x3/1	26x26x128	26x26x128
21	conv	128	3x3/1	26x26x128	26x26x128
22	route 21, 20				26x26x256
23	conv	256	1x1/1	26x26x256	26x26x256
24	route 18, 23				26x26x512
25	max		2x2/2	26x26x512	13x13x512
26	conv	512	3x3/1	13x13x512	13x13x512
27	conv	256	1x1/1	13x13x512	13x13x256
28	conv	512	3x3/1	13x13x256	13x13x512
29	conv	255	1x1/1	13x13x512	13x13x255
30	yolo				
31	route 27				13x13x256
32	conv	128	1x1/1	13x13x256	13x13x128
33	upsample		2x	13x13x128	26x26x128
34	route 33, 23				26x26x384
35	conv	256	3x3/1	26x26x384	26x26x256
36	conv	255	1x1/1	26x26x256	26x26x255
37	yolo				
38	detection				

Fonte: Darknet

2.7 Reconhecimento de Caracteres

É utilizado um método de segmentação quando palavras inteiras são analisadas. De acordo com Clemente, Rodriguez e Gay (2002), dada uma palavra o algoritmo a separa em n números de caracteres (sendo n o número de elementos que compõem a palavra), para comparar esses dígitos com o alfabeto que a máquina tem na memória, fazendo o reconhecendo de cada letra, mas não o significado das palavras.

Existem dois tipos principais de Sistemas de reconhecimento:

- Sistemas de reconhecimento de caracteres isolados, que consistem em analisar imagens que contém apenas um caractere, tornando uma etapa de segmentação desnecessária.

- Sistemas de reconhecimento de escrita, que analisa palavras completas separando os elementos das palavras e os reconhecendo individualmente. Esse sistema depende do método de segmentação, o que o torna um processo mais lento e custoso.

A seguir em 2.7.1 e 2.7.2 será apresentado duas técnicas que fazem o reconhecimento de caracteres.

2.7.1 Reconhecimento Óptico de Caracteres

De acordo com Mithe, Indalkar e Divekar (2013), o OCR (*Optical Character Recognition* - Reconhecimento Ótico de Caracteres) permite o reconhecimento de caracteres através de um mecanismo óptico. No caso humano o mecanismo é o olho, mas considerando uma máquina o mecanismo utilizado é uma câmera que pode captar imagens sejam elas vídeo ou fotos.

O OCR pode ser comparado com a habilidade humana da leitura, mesmo não sendo tão eficiente ele consegue reconhecer textos manuscritos e digitais. No entanto o OCR depende muito da qualidade da sua imagem de entrada, uma vez que ele foi desenvolvido para processar imagens compostas inteiramente por texto, ou seja, o fundo da imagem geralmente interfere e dificulta o processo de reconhecimento.

2.7.2 Tesseract

Conforme Smith (2007), Tesseract é uma *engine* OCR *open source* pertencente à Google e desenvolvida pela HP (Hewlett-Packard, companhia de tecnologia da informação multinacional americana). O funcionamento dele baseia-se em um sistema independente de disposição de página, logo quando recebe uma

imagem, ele a transforma em uma imagem binária, com regiões de texto poligonais opcionais definidos.

A *engine* destaca caracteres e junta diversos destacados para formar amontoados de caracteres. Esses amontoados são organizados em linhas de texto, e as linhas e regiões são analisados e separados em textos fixos e textos proporcionais. Então as linhas de texto são divididas em palavras considerando o espaçamento dos caracteres. Esses textos fixos são cortados imediatamente por células de caracteres. O texto proporcional é dividido em palavras utilizando espaços definidos e espaços diferenciados.

O reconhecimento prossegue em dois passos, o primeiro consiste em uma tentativa de reconhecer cada palavra por turnos. Cada palavra satisfatória é passada para um classificador adaptativo como dados de treinamento. O classificador tem a chance de reconhecer mais palavras de maneira mais correta no restante do texto.

É possível que o classificador tenha aprendido algo muito tarde em seu ciclo de ativação, logo o segundo passo consiste em analisar novamente a página considerando as palavras que não foram corretamente reconhecidas na primeira parte.

A parte final resolve os espaços estranhos e checa hipóteses diferentes para um tamanho n, com finalidade de localizar textos em minúsculo. Todas essas partes do processo Tesseract estão representadas na Figura 15, que ilustra o fluxo da arquitetura Tesseract. Durante esse processo são implementados os seguintes algoritmos:

- **Encontrar linhas**: O algoritmo é montado de maneira que, uma página com linhas tortas não tenha que ser desentortada, impedindo uma perda de qualidade de imagem. Assumindo que a análise do *layout* da página proveu regiões de texto com tamanhos uniformes, é passado um filtro de tamanho simples, eliminando alguns caracteres especiais. De acordo com o tamanho médio é possível eliminar amontoados menores, que são compostos por pontuações. Os amontoados de textos filtrados tem probabilidade maior de se encaixarem em um molde de linha de texto. O último passo consiste em fundir os amontoados de texto que são no mínimo 50% similares, associando partes

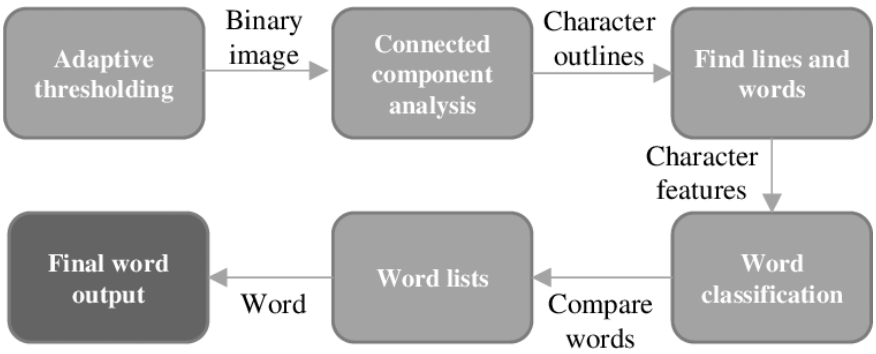
de caracteres quebrados;

- **Ajustar as bases:** Com as linhas encontradas, é utilizado um *spline* (curva definida matematicamente por dois ou mais pontos de controle) quadrático para ajustar suas linhas de base. Essas linhas são ajustadas por meio de uma partição dos amontoados de texto em grupos com deslocamento continuo em relação a linha de base inicial. O *spline* é utilizado na partição mais populosa;

- **Arrumar os textos fixos e recortes:** O Tesseract testa as linhas para determinar se elas têm textos fixos. Quando ele encontra os textos fixos, as palavras são divididas em caracteres e são associados a essas palavras na fase de reconhecimento;

- **Encontrar textos proporcionais:** O Tesseract resolve este problema medindo os espaços limitados pelo espaço vertical entre a linha imaginária no topo da altura x e a linha imaginária na base da altura x. Espaços muito perto dos limites são determinados como confusos, postergando a decisão final para depois da identificação das palavras.

Figura 15 - Arquitetura Tesseract OCR engine



Fonte: Gjoresk (2020)

2.8 Conjunto de dados UFPR-ALPR

De acordo com UFPR-ALPR... (2018), este conjunto de dados (*dataset*) denominado conjunto de dados UFPR-ALPR, inclui 4.500 imagens totalmente anotadas (mais de 30.000 caracteres LP) de 150 veículos em cenários do mundo real onde o veículo e a câmera (dentro de outro veículo) estão se movendo.

As imagens foram adquiridas com três câmeras diferentes e estão disponíveis no formato PNG (*Portable Network Graphics*) com tamanho de 1.920 × 1.080 pixels. As câmeras usadas foram: GoPro Hero4 Silver, Huawei P9 Lite e iPhone 7 Plus. Foram coletadas 1.500 imagens com cada câmera, divididas da seguinte forma:

- 900 de carros com placa cinza;
- 300 carros com placa vermelha;
- 300 motocicletas com placa cinza.

Cada imagem possui as seguintes anotações disponíveis em um arquivo de texto: a câmera em que a imagem foi tirada, a posição do veículo e informações como tipo (carro ou motocicleta), fabricante, modelo e ano; a identificação e posição da placa, bem como a posição de seus caracteres.

Figura 16 - Exemplo de imagens do dataset UFPR-ALPR

Fonte: UFPR-ALPR... (2018)

2.9 Trabalhos Relacionados

No trabalho de Gonçalves, Menotti e Schwartz (2016), apresentam uma abordagem em tempo real para detectar automaticamente veículos e identificar reconhecer sua placa com base em informações de redundância temporal. Além de proporem duas técnicas de pós-processamento consultando um banco de dados de placas. Os resultados, baseados em um *dataset* de 300 veículos em situações normais da estrada urbana, demonstram que é possível incrementar a porcentagem de reconhecimento em 15,3% através da redundância temporal. Uma porcentagem adicional de 7,8% foi atingido consultando placas registradas em um banco de dados pela ordem de aparição do veículo, levando a um resultado final de reconhecimento de 89,6%. A técnica também é capaz de processar a 34 quadros por segundo, o que a caracteriza como uma abordagem em tempo real.

No trabalho de Peixoto, Gonçalves, Chávez, Menotti e Schwartz (2015), são apresentadas diferentes abordagens de rede convolucional com o objetivo de melhorar as representações de característica de imagens de caracteres utilizando *deep learning*. Foram realizados experimentos em um *dataset* de placas brasileiras,

o *Sense License Plate Character Segmentation Database* (SSIG) que possui aproximadamente 8 Gigabytes de caracteres segmentados de placas veiculares. A partir dos resultados do trabalho foi possível observar que a abordagem utiliza duas estratégias. O primeiro é algoritmo de *back-propagation* para aprender os pesos dos filtros utilizando data augmentation. A segunda é a estratégia de agregação de camada localmente conectada à CNN, a qual obteve 99.83% de precisão para letras. Entretanto, para o tratamento de dígitos, a acurácia utilizada é a abordagem que emprega filtros aleatórios para melhorar a arquitetura, o qual resultou em 99,81% de precisão.

No trabalho de Laroca et al. (2018), é apresentado um sistema ALPR criado com base em um detector de objetos YOLO. São criados dois modelos de rede convolucionais, um deles detecta o carro enquanto o outro é treinado para captar a placa automobilística em diferentes condições de iluminação, posicionamento e outras. Por padrão o YOLO retorna apenas objetos detectados com uma confiança acima de 25%, mas foi para a detecção de placas foi escolhido um *threshold* de 0 para incluir os casos com pouca confiança, quando ocorrem casos de múltiplas placas detectadas é escolhida aquela com maior confiança. Foram utilizadas também duas redes convolucionais para o reconhecimento de caracteres, com a mesma técnica utilizada para detecção de placas, com o *threshold* sendo 0. O resultado da implementação dessas técnicas é uma taxa de detecção de mais de seis caracteres de 87.33% com 64.89% acertos com todos corretos, quando utilizado com redundância temos respectivamente comparado aos apresentados anteriormente, resultados de 88.33% e 78.33%. Os pontos focais do trabalho são descritos como um sistema de dois estágios para a segmentação e reconhecimento de caracteres, criação do *dataset* UFPR-ALPR e por fim a comparação com versões anteriores do projeto e dois sistemas comerciais.

No trabalho de Laroca et al. (2019), apresentam um ALPR baseado no detector de objetos de ultima geração YOLO, que contém uma abordagem unificada para a detecção da placa automobilística e para a classificação do *layout* em função da melhora do reconhecimento utilizando as regras de pós processamento. O sistema foi criado avaliando e otimizando outros modelos com diversas modificações a fim de alcançar a melhor velocidade/acurácia em cada estágio. O sistema proposto alcançou uma média de reconhecimento de 96,8% através de 8 *datasets* públicos

(cinco de diferentes regiões) usados em experimentos, se provando melhor do que versões anteriores e sistemas comerciais nos *datasets:* ChineseLP, OpenALPR-EU, SSIG-SegPlate e UFPR-ALPR. Nos outros casos, o sistema apresentou resultados competitivos.

3 Desenvolvimento

Este capítulo apresenta descrição da proposta e o desenvolvimento do sistema de reconhecimento automático de placas automotivas brasileiras (modelos anteriores àquele padronizado pelo Mercosul).

3.1 Proposta da Solução

O presente trabalho de conclusão de curso propõe a implementação de um sistema de detecção e reconhecimento de placas automotivas brasileiras (modelos anteriores àquele padronizado pelo Mercosul).

A linguagem de programação Python foi escolhida para o desenvolvimento do sistema devido a sua praticidade e fácil interpretação, além de possuir uma grande disponibilidade de bibliotecas e artigos acadêmicos na área de Redes Convolucionais e de Inteligência Artificial.

O processo de desenvolvimento foi separado em cinco etapas:

1. Para detectar Placas de Identificação Veiculares brasileiras antigas, foi necessário criar um modelo de aprendizado de máquina personalizado, o YOLOv4 Tiny. Essa etapa de criação foi dividida em duas partes. A primeira parte consistiu em anotar as 4500 placas do *dataset* UFPR-ALPR. A segunda foi a parte de criação da rede, onde foi feita a configuração e o re-treinamento da rede YOLOv4 Tiny para a detecção de placas veiculares;

1.1. Utilizando o programa chamado LabelImg criou-se um arquivo de suporte do tipo texto (.txt) para cada imagem do *dataset* que contenha duas informações principais: as coordenadas em um plano cartesiano (x inicial, y inicial, x final, y final) das placas automobilísticas e a classe a qual elas pertencem como objeto, que no caso foi placa veicular (*license plate*). Essa etapa é necessária, pois a forma que as anotações que os criadores criaram do *dataset* não são compatíveis com a forma que o treinamento da rede

exige;

1.2. Para re-treinamento da rede neural convolucional YOLOv4 Tiny, foi utilizado o *framework* Darknet com intuito de configurar o número de *batches*, número máximo de *baches*, quantidade de classes, quantidade de *subdivisions*, tamanho da rede e outros parâmetros.

2. O modelo YOLO foi alimentado com o *dataset* UFPR-ALPR tratado, no qual será separado em 90% das imagens para treinamento e 10% para testes de validação. Com o modelo pronto em mãos, foi possível prosseguir para os próximos passos, dos quais iniciaram-se o processo de desenvolvimento da aplicação;

3. Com o modelo YOLOv4 Tiny treinado, sendo capaz de reconhecer placas em imagens e retornar sua devida localização cartesiana, iniciou-se então o pré processamento da imagem com intuito de reconhecer seus caracteres. Nessa etapa utilizou-se a biblioteca OpenCV, onde a imagem foi submetida para recorte da identificação do veículo e aplicação de diversas operações, como: de suavização de imagem, transformações morfológicas e geométricas com objetivo de segmentar corretamente cada caractere da placa veicular;

4. Nesta etapa, utilizou-se o Tesseract, que é uma OCR capaz de reconhecer textos a partir de imagens como mencionado na seção 2.7.2, e consequentemente extrair individualmente cada caractere segmentado da placa de identificação veicular. Para isso, utilizou-se a biblioteca PyTesseract, no qual é um *wrapper* que permite a utilização do Tesseract em linguagem de programação Python;

5. Para as devidas validações, realizou-se testes com 10% do dataset UFPR-ALPR composto de 4500 imagens separado previamente, ou seja 450 imagens diferentes daquelas que foram colocadas no treinamento da rede. Os resultados finais dessa validação foram comparados com resultados de outros trabalhos que utilizam técnicas similares, como aqueles citados na seção 2.9.

Como por exemplo, o trabalho de Laroca et al. (2018) que obteve recall de 98,33% na deteção de placas veiculares e precisão de reconhecimento de caracteres de 90,37%.

3.2 Técnicas utilizadas

Para realizar o trabalho foram utilizadas diversas técnicas de visão computacional, aprendizado de máquina e processamento de imagens. Seguem as soluções escolhidas:

- Para detecção de placas automobilísticas, foi escolhida a rede neural convolucional YOLOv4 Tiny para re-treinamento utilizando o *dataset* UFPR-ALPR.

- Para pré-processamento de imagens, foi escolhida a biblioteca de visão computacional OpenCV com intuito de utilizar operações como trocas de ambientes de cores, dimensionamento, limitação simples, limitação adaptativa, erosão e etc.

- Para o reconhecimento dos caracteres da placa veicular, foi escolhida a *engine* OCR *open source* Tesseract, em conjunto com a biblioteca PyTesseract, que é considerado um *wrapper* do Tesseract, ou seja, pode ser considerado um interpretador do Tesseract original, servindo como a ponte entre o programa escrito na linguagem Python e o Tesseract que é escrito na linguagem C. Ele também pode ser utilizado como um *script* de invocação independente, uma vez que pode ser utilizado para ler todos os tipos de imagens suportadas pelas bibliotecas de imagem Pillow e Leptonica, incluindo jpeg, png, gif, bmp, tiff, e outros

- Para pós-processamento de imagens, foi criado um algoritmo que verifica a estrutura da string retornada pelo algoritmo de reconhecimento de caracteres, e substitui devidamente aqueles que foram reconhecidos de forma incorreta em strings com tamanho 7.

3.3 Conjunto de Dados

Os dados aplicados no desenvolvimento desse trabalho foram coletados por arquivos de dados abertos em repositórios de livre acesso. Para re-treinamento do modelo, foi utilizado o *dataset* da Seção 2.9.

3.3.1 UFPR-ALPR Dataset

Segundo a Universidade Federal do Paraná, o *dataset* o UFPR-ALPR é composto por 4500 imagens de 150 veículos, variando entre carros, motos, ônibus e caminhões, dispostos em cenários do mundo real, todos possuindo a placa veicular do modelo antigo.

O *dataset* foi manipulado em uma proporção de 90% (4050 imagens) para treino e 10% (450 imagens) para testes. Esse *dataset* está disponível apenas para fins acadêmicos.

3.4 Materiais

Para implementação do programa proposto, foi utilizada a linguagem de programação Python, na versão 3.6.5, disponível no site oficial da linguagem. Ela foi escolhida devido ao montante de trabalhos disponíveis na área de Inteligência Artificial, algoritmos de ciências de dados e aprendizagem computacional. Além disso, a linguagem oferece desempenho equiparável com outras de baixo nível, devido a sua compatibilidade principalmente com bibliotecas escritas em linguagem C.

Além do Opencv, este trabalho utiliza três bibliotecas que podem ser destacadas, Numpy, PyTesseract e Csv.

O Numpy é uma biblioteca utilizada principalmente para realizar cálculos em vetores multi-dimensionais. O PyTesseract é uma biblioteca que serve como interprete para o Tesseract. E por fim, o Csv é uma biblioteca que permite e facilita a leitura e escrita de arquivos .csv.

É utilizado um *software* chamado LabelImg criado por LIN (2015), que permite que o usuário crie anotações de imagens compatíveis com a forma que o treinamento de redes neurais convolucionais YOLO exige. Quando esse *software* é utilizado, é criado um arquivo de suporte do tipo texto (.txt) para cada imagem informada, contendo as coordenadas da *bounding box* marcada pelo usuário no LabelImg.

E por fim, o TensorFlow é uma plataforma de código aberto para aprendizado de máquina. Ele tem um ecossistema de ferramentas, bibliotecas e recursos da comunidade que permite desenvolvedores criar e implantar aplicativos com tecnologia de aprendizado de máquina.

Bochkovskiy et al. (2020), criadores do YOLO, recomendam TensorFlow caso os desenvolvedores desejem trabalhar com YOLO utilizando a linguagem de programação Python, pois segundo TensorFlow (2020) é possível converter os pesos (.weights) da rede desejada em .pb (formato compatível com TensorFlow) utilizando o formato .savedmodel.

3.5 Especificações técnicas

Especificações do computador utilizado para o **treinamento** da rede convolucional:
Processador: Intel Core i7-8700K 3.70GHz Hexa-Core
Memória RAM: 16 GB 2666 MHz DDR4
Placa de Vídeo: Gygabite GTX 1070TI 6 GB
Sistema Operacional: Windows 10

Especificações dos computadores utilizados para os **testes** do sistema:
MacBook Pro (Retina, 15-inch, Mid 2015)
Processador: Intel Core i7 2.2 GHz Quad-Core
Memória RAM: 16 GB 1600 MHz DDR3
Placa de Vídeo: Intel Iris Pro 1536 MB
Sistema Operacional: MacOS Catalina

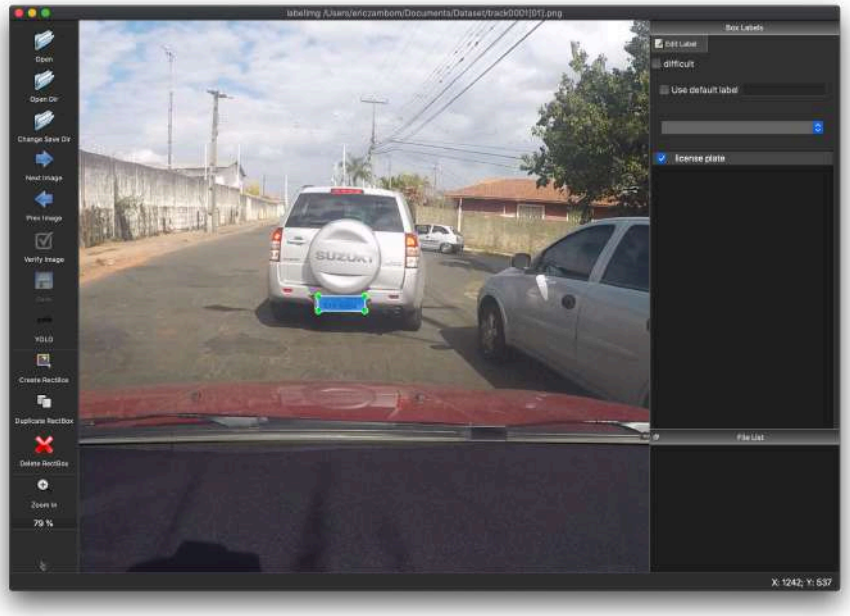
3.6 Desenvolvimento do Sistema

3.6.1 Criação do *dataset* ou anotando as imagens

Com a obtenção do *dataset* para treinamento da CNN, o UFPR-ALPR, o mesmo foi separado em pastas onde cada pasta continha 30 imagens de um veículo seguindo em uma via, além de uma anotação com as informações da imagem em um arquivo de texto (.txt), isso se repetindo 150 vezes.

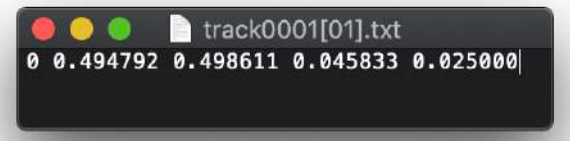
Apesar de cada imagem ter sua anotação individual, seu formato não era compatível com o requisitado pela treinamento da CNN. Dessa forma, utilizando o *software* chamado LabelImg, ilustrado na Figura 17, são criadas novas anotações no formato YOLO, como mostrado na Figura 18, para as 4500 imagens do *dataset*.

Figura 17 - Processo de criação das anotações



Fonte: Própria

Figura 18 - Anotação de uma imagem



Fonte: Própria

Na Figura 18 a anotação consiste das informações: um número que indica a classe do objeto detectado, e em seguida considerando um plano cartesiano com com 2 dimensões temos os valores: valor inicial X, valor inicial Y, valor final X e valor final Y para indicar as coordenadas da placa no plano.

3.6.2 Configurando e treinando a rede YOLOv4 Tiny

Para realizar a tarefa de reconhecimento de placas, foi escolhida a rede neural convolucional YOLOv4 para re-treinamento utilizando 90% das imagens escolhidas aleatoriamente do *dataset* UFPR-ALPR, como dito previamente. E para garantir que a rede convolucional YOLO fosse treinada de acordo com as recomendações dos criadores, Bochkovskiy et al. (2020), o arquivo de configuração (.cfg) da CNN foi modificado, alterando o comportamento de certas camadas da rede e configurações de treinamento. A seguir será apresentado algumas das alterações principais que foram realizadas:

- ***batch* de 64**, define o número de imagens que serão carregadas em uma iteração;
- ***subdivisions* de 16**, determina o número de imagens que serão analisadas em paralelo dividindo o batch por esse número e garantido uma velocidade maior;
- **Largura da rede de 608 pixels**, a rede reduz a largura da entrada (no caso uma imagem) para 608 pixels;

- **Altura da rede de 608 pixels**, a rede reduz a altura da entrada (no caso uma imagem) para 608 pixels;
- **Aleatório ativado nas camadas YOLO**, para garantir a otimização da alocação de memória durante o redimensionamento da rede, além de aumentar a precisão da rede para diferentes resoluções;
- **Filtros de 18 nas camadas convolucionais anteriores as camadas YOLO**, sendo essa a quantidade de classes do modelo, somado com cinco e multiplicado por três (1 + 5 * 3 = 18);
- **Quantidade de classes/objetos** a serem reconhecidas pela rede, na qual o valor será somente 1.
- **Quantidade máxima de *batches*** para 15000;
- E por fim, a **quantidade de passos** para 12000, 13500 (80% e 90% da qualidade máxima de *batches,* respectivamente)

As modificações acima foram as mais "balanceadas" dentre os dois treinamentos realizados do modelo.

O primeiro treinamento englobou todas as alterações descritas acima, com exceção da "Aleatório ativado nas camadas YOLO”, no que resultou em um recall de treinamento de 71%, na qual não foi satisfatório.

O segundo treinamento conteve todas as alterações descritas acima, no que resultou em um recall de treinamento de 98,81%, deixando de detectar somente 53 placas de motocicletas ou carros, em cenários que a imagem não continha boa resolução ou estava ofuscada, e a placa automobilística estava com sombras ou danificada. A Figura 19 ilustra um caso que nenhuma placa foi detectada pela rede, no qual a placa da moto deveria ser reconhecida.

Figura 19 - Imagem exemplo de caso que nenhuma placa foi detectada pela rede

Fonte: UFPR-ALPR

A seguir, a Tabela 5 apresenta a arquitetura das duas redes descritas.

Tabela 5 - Arquitetura da YOLOv4 Tiny para detecção de placas veiculares

Layer	***Type***	***Filters***	***Size/Stride***	***Input***	***Output***
0	conv	32	3x3/2	608x608x3	304x304x32
1	conv	64	3x3/2	304x304x32	152x152x64
2	conv	64	3x3/1	152x152x64	152x152x64
3	route 2			1/2	152x152x32
4	conv	32	3x3/1	152x152x32	152x152x32
5	conv	32	3x3/1	152x152x32	152x152x32
6	route 5, 4				152x152x64
7	conv	64	1x1/1	152x152x64	152x152x64
8	route 2, 7				152x152x128
9	max		2x2/2	152x152x128	76x76x128
10	conv	128	3x3/1	76x76x128	76x76x128
11	route 10			1/2	76x76x64
12	conv	64	3x3/1	76x76x64	76x76x64
13	conv	64	3x3/1	76x76x64	76x76x64
14	route 13, 12				76x76x128

15	conv	128	1x1/1	76x76x128	76x76x128
16	route 10, 15				76x76x256
17	max		2x2/2	76x76x256	38x38x256
18	conv	256	3x3/1	38x38x256	38x38x256
19	route 18			1/2	38x38x128
20	conv	128	3x3/1	38x38x128	38x38x128
21	conv	128	3x3/1	38x38x128	38x38x128
22	route 21, 20				38x38x256
23	conv	256	1x1/1	38x38x256	38x38x256
24	route 18, 23				38x38x512
25	max		2x2/2	38x38x512	19x19x512
26	conv	512	3x3/1	19x19x512	19x19x512
27	conv	256	1x1/1	19x19x512	19x19x18
28	conv	512	3x3/1	19x19x256	19x19x512
29	conv	18	1x1/1	19x19x512	19x19x18
30	yolo				
31	route 27				19x19x256
32	conv	128	1x1/1	19x19x256	19x19x128
33	upsample		2x	19x19x128	38x38x128
34	route 33, 23				38x38x384
35	conv	256	3x3/1	38x38x384	38x38x256
36	conv	18	1x1/1	38x38x256	38x38x18
37	yolo				

Fonte: Própria

Cada treinamento durou em média 15 horas para ser concluído, totalizando no mínimo 30 horas de treino.

3.6.3 Convertendo o modelo para TensorFlow

Foi utilizado como base do projeto o repositório *open source* e de uso gratuito, criado por HUNG (2020) e recomendado pelos criadores do YOLOv4 Bochkovskiy et al. (2020) caso o desenvolvedor queira utilizar a linguagem de pogramação Python ao invés da linguagem C.

O repositório tem como proposta dois objetivos. O primeiro, a conversão de pesos (.weights) de modelos Darknet YOLO em TensorFlow (.pb), TensorFlow Lite (.tflite) e TensorRT (.trt). E o segundo, disponibilizar uma plataforma para testes de deteção a partir de modelos TensorFlow YOLO.

Com finalidade de converter os pesos foi aplicado o algoritmo presente no arquivo *save_model.py*, que recebe como parâmetros de entrada:

- O caminho do arquivo do tipo .weights;
- O caminho da configuração da CNN (.cfg);
- O tipo do modelo YOLO de entrada, ou seja, se o modelo é Tiny ou não;
- O tipo do modelo de saída, ou seja, se será TensorFlow, TensorFlow Lite ou TensorRT.

A partir deles, o algoritmo faz a decodificação da configuração e gera o novo modelo no formato desejado, que no caso desta monografia foi utilizado o formato TensorFlow (.pb).

3.6.4 Pré-processamento das imagens

Uma vez que o modelo é treinado e convertido, o TensorFlow YOLOv4, é capaz de retornar a localização cartesiana de uma placa de veicular a partir de uma imagem de entrada, segue-se para uma fase algorítmica de pré-processamento, que consiste em melhorar aspectos de uma imagem para torná-la mais adequada para extração dos caracteres.

Para esse fim utiliza-se o OpenCV, que possui uma grande aglomeração de funções e utilidades prontas. Dessa forma, foram criados 7 filtros de imagem, onde eles priorizam diferentes aspectos das imagens apresentadas, com parâmetros variantes e misturas de técnicas diferentes. Essa abordagem de múltiplos filtros foi utilizada com o objetivo de aumentar o número de placas identificadas de maneira correta, e o número caracteres por placa, já que o *dataset* possui imagens com diferentes cenários de captura.

As principais funções de OpenCV aplicadas nesses diversos filtros para imagem são:

- Trocas de ambientes de cores;
- Redimensionamento;

- Suavização de imagem;
- Limitação simples;
- Limitação Adaptativa;
- Erosão;
- Dilatação.

Todos os filtros iniciam a partir da imagem da placa recortada, sendo assim os 7 filtros são:

- O Filtro 1 está dividido em 5 etapas como descrito a seguir:
  1. Aplicar uma função que troca o ambiente de cores da imagem para escala de cinza;
  2. Redimensionar a imagem em uma escala de 3x;
  3. Aplicar suavização Gaussiana com kernel de altura 5 e largura 5;
  4. Aplicar limitação simples de 0 à 255, utilizando limitação OTSU e limitação binária invertida;
  5. Aplicar dilatação com elemento estruturante de tamanho 3 por 3.

- O Filtro 2, por sua vez, é composto por 9 etapas, que são:
  1. Aplicar uma função que detecta tonalidades de cinza de RGB (59, 59, 59) à RGB (255, 255, 255) do plano de fundo de uma imagem e as substitui por RGB (255, 255, 255);
  2. Aplicar uma função que troca o ambiente de cores da imagem para escala de cinza;
  3. Redimensionar a imagem em uma escala de 3x;
  4. Aplicar uma função que melhora o brilho e o gamma da imagem;
  5. Aplicar suavização Gaussiana com kernel de altura 5 e largura 5;
  6. Aplicar limitação simples de 127 à 255, utilizando limitação OTSU e limitação TRUNC;
  7. Aplicar limitação simples de 0 à 255, utilizando limitação OTSU e limitação binária invertida;
  8. Aplicar erosão com elemento estruturante de tamanho 3 por 3;
  9. Aplicar dilatação com elemento estruturante de tamanho 3 por 3.

- O Filtro 3, é construído com 6 etapas, e são elas:
  1. Aplicar uma função que troca o ambiente de cores da imagem para escala de cinza;
  2. Redimensionar a imagem em uma escala de 3x;
  3. Aplicar suavização Gaussiana com kernel de altura 5 e largura 5;
  4. Aplicar limitação simples de 127 à 255, utilizando limitação OTSU e limitação TRUNC;
  5. Aplicar limitação adaptativa Gaussiana, utilizando limitação binária invertida com tamanho de vizinhança 41 e constante 4;
  6. Aplicar dilatação com elemento estruturante de tamanho 5 por 5.

- O Filtro 4, também apresenta 6 etapas, sendo elas:
  1. Aplicar uma função que troca o ambiente de cores da imagem para escala de cinza;
  2. Redimensionar a imagem em uma escala de 4x;
  3. Aplicar suavização Gaussiana com kernel de altura 5 e largura 5;
  4. Aplicar limitação adaptativa Gaussiana, utilizando limitação binária invertida com tamanho de vizinhança 51 e constante 10;
  5. Aplicar erosão com elemento estruturante de tamanho 5 por 5;
  6. Aplicar dilatação com elemento estruturante de tamanho 5 por 5.

- O Filtro 5, funciona com o mesmo número de etapas, eles são:
  1. Aplicar uma função que troca o ambiente de cores da imagem para escala de cinza;
  2. Redimensionar a imagem em uma escala de 4x;
  3. Aplicar suavização Gaussiana com kernel de altura 5 e largura 5;
  4. Aplicar limitação adaptativa Gaussiana, utilizando limitação binária invertida com tamanho de vizinhança 121 e constante 16;
  5. Aplicar erosão com elemento estruturante de tamanho 5 por 5;
  6. Aplicar dilatação com elemento estruturante de tamanho 5 por 5.

- O Filtro 6, composto por 6 etapas, que apresentam:
  1. Aplicar uma função que troca o ambiente de cores da imagem para escala de cinza;
  2. Redimensionar a imagem em uma escala de 4x;
  3. Aplicar suavização Gaussiana com kernel de altura 5 e largura 5;
  4. Aplicar limitação adaptativa Gaussiana, utilizando limitação binária invertida com tamanho de vizinhança 121 e constante 16;
  5. Aplicar erosão com elemento estruturante de tamanho 3 por 3;
  6. Aplicar dilatação com elemento estruturante de tamanho 3 por 3.

- O Filtro 7 é o ultimo filtro e apresenta 6 etapas, sendo elas:
  1. Aplicar uma função que troca o ambiente de cores da imagem para escala de cinza;
  2. Redimensionar a imagem em uma escala de 4x;
  3. Aplicar suavização Gaussiana com kernel de altura 5 e largura 5;
  4. Aplicar limitação simples de 127 à 255, utilizando limitação OTSU e limitação TRUNC;
  5. Aplicar limitação simples de 127 à 255, utilizando limitação OTSU e limitação bináriar invertida;
  6. Aplicar dilatação com elemento estruturante de tamanho 3 por 3.

Como exemplo do funcionamento de um desses filtros, é apresentado o funcionamento completo do Filtro 6, cobrindo todas as operações.

Figura 20 - Imagem da placa recortada após ser reconhecida pelo modelo

Fonte: UFPR-ALPR (2018)

O Filtro 6 começa recebendo o recorte somente da placa do veículo, como mostrado na Figura 20.

Figura 21 - Exemplo do resultado da etapa 1 do Filtro 6

Fonte: UFPR-ALPR (2018)

A Figura 21 apresenta o resultado da etapa 1 do Filtro 6, com a troca de ambientes de cores da imagem em variações de cinza.

Figura 22 - Exemplo do resultado da etapa 2 do Filtro 6

Fonte: UFPR-ALPR (2018)

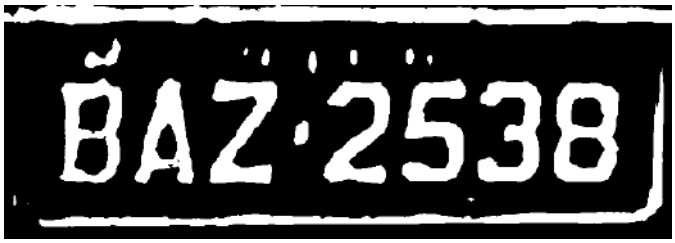
A Figura 22 mostra o redimensionamento da imagem em uma escala de 4 vezes o tamanho original do recorte.

Figura 23 - Exemplo do resultado da etapa 3 do Filtro 6

Fonte: UFPR-ALPR (2018)

A Figura 23 ilustra a aplicação da suavização gaussiana, com kernel de altura 5 e largura 5.

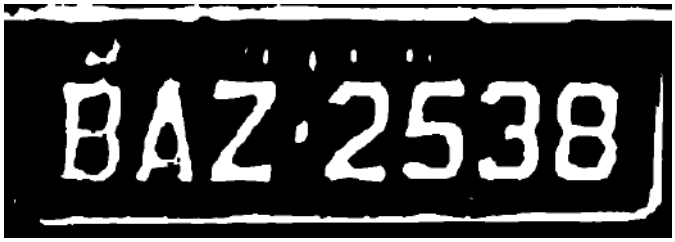
Figura 24 - Exemplo do resultado da etapa 4 do Filtro 6



Fonte: UFPR-ALPR (2018)

A Figura 24 apresenta a aplicação da limitação adaptativa gaussiana, com limitação binária invertida com tamanho de vizinhança 121 e constante 16.

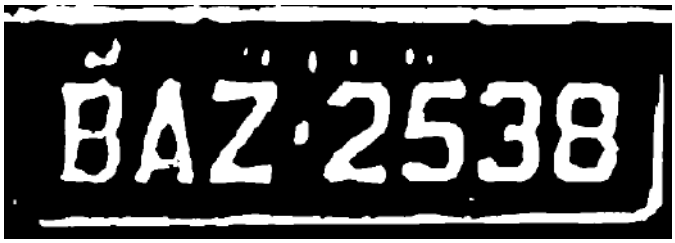
Figura 25 - Exemplo do resultado da etapa 5 do Filtro 6



Fonte: UFPR-ALPR (2018)

Na Figura 25, é mostrado o resultado da função de erosão com elemento estruturante de tamanho 3 por 3.

Figura 26 - Exemplo do resultado da etapa 7 do Filtro 6



Fonte: UFPR-ALPR (2018)

A Figura 26, mostra a última etapa do processo, o resultado da função de dilatação com elemento estruturante de tamanho 3 por 3.

Obtendo o resultado de todos os 7 filtros, o programa está pronto para realizar a segmentação dos caracteres apresentados nas imagens resultado, e inicializar o Tesseract para o reconhecimento do texto.

3.6.5 Reconhecendo os caracteres da placa automobilística

Como descrito na seção 3.2, a técnica utilizada para reconhecer os caracteres foi a aplicação do Tesseract OCR a partir da segmentação dos caracteres da placa. Dessa forma, essa etapa pode ser divida em duas partes, a de segmentação e a de reconhecimento.

Primeiramente, com objetivo do Tesseract reconhecer individualmente cada caractere da placa, é realizada a segmentação dos caracteres utilizando a operação da biblioteca OpenCV que procura contornos em uma imagem binária. Essa operação recebe como parâmetros de entrada:

- A imagem binarizada da placa;
- RETR_TREE como modo de recuperação de contorno, na qual recupera todos os contornos e reconstrói uma hierarquia completa de contornos aninhados;
- CHAIN_APPROX_SIMPLE como método de aproximação de contorno, no qual comprime segmentos horizontais, verticais e diagonais e deixa apenas seus pontos finais.

A função procura retornos tem como saída um vetor contendo coordenadas dos contornos encontrados, dessa forma, é executada a função *sort* da linguagem de programação Python para ordenar os contornos encontrados do menor valor da coordenada X para maior valor da coordenada X, com intuito de reconhecer a placa veicular da esquerda para a direita.

Com objetivo de detectar individualmente o conteúdo dos contornos, o vetor de contornos é percorrido através de um laço de repetição. Cada iteração do laço contém a coordenada de um contorno, e partir disso, é feito um corte na imagem binária da placa automobilística utilizando essa coordenada. Em seguida, com essa imagem cortada contendo um possível caractere, como ilustrado na Figura 27, o PyTesseract tenta reconhecer seu texto.

Figura 27 - Imagem de um caractere segmentado de uma placa veicular

Fonte: Própria

Para reconhecer os caracteres o PyTesseract é configurado com os seguintes parâmetros:

- PSM com valor 10, parâmetro que define qual o tipo de deteção o OCR fará, que no caso com valor 10, segundo ABDULKADER et al (2020), muda o comportamento dele para reconhecer imagens com apenas um caractere, restringindo múltiplas detecções ou tentativas de reconhecer palavras;
- OEM com valor 3, parâmetro que define o modo de operação da OCR, quando aplicado com esse valor define que o OCR irá utilizar o modo padrão.
- *Whitelist* com intervalo de A a Z e 0 a 9, parâmetro que restringe as possibilidades de reconhecimento.

Após a biblioteca retornar o resultado encontrado, o resultado é concatenado com os outros resultados obtidos e esse processo se repete até o último contorno do vetor.

Quando esse processo acaba, a string com todos os caracteres reconhecidos é tratada utilizando as funções da biblioteca Python para retirar possíveis espaços inseridos pelo algoritmo de reconhecimento.

3.6.6 Pós-processamento das imagens

No pós processamento é criada uma verificação para certificar de que a placa detectada e lida tem sete caracteres na formatação correta, sendo ela três letras seguidas de quatro números. Para isso o algoritmo verifica o tamanho da *string* que o Tesseract retorna, se o tamanho for igual a sete a *string* é dividida em duas parte, os três primeiros caracteres da *string* e os quatro últimos. São utilizadas as funções

do Python que checam se os caracteres apresentados na string são todos letras e se são todos números, retornando um valor lógico verdadeiro ou falso. Caso ele retorne como falso é determinado que ocorreu um erro de reconhecimento e a placa de identificação tem no mínimo um falso positivo.

Foram identificados alguns casos em que números e letras eram confundidos frequentemente pelo OCR, para consertar esses erros e melhorar o desempenho do nosso sistema foi criada uma função que checa os caracteres que foram reconhecidos erroneamente e substitui por seus correspondentes. Os casos de letras são:

- Substituir 1 por I
- Substituir 2 por Z
- Substituir 5 por S
- Substituir 8 por B
- Substituir 0 por O

Já os casos de números são:

- Substituir O ou Q por 0
- Substituir B por 8
- Substituir G por 9
- Substituir Z por 2
- Substituir S por 5

Com essas substituições feitas a função termina sua operação. Com o término do pós processamento, são comparados os resultados dos filtros, sendo selecionado aquele que tem o maior número de caracteres identificados.

4 Resultados e discussões

Este capítulo apresenta as descrições dos testes realizados, seus resultados e a análise destes resultados. Os resultados são discutidos, com base nas informações dos capítulos anteriores.

4.1 Métodos de Avaliação

Com o intuito de validar o trabalho desenvolvido, foi necessária a escolha de uma metodologia de avaliação. Dessa forma, para analisar os aspectos do sistema foram utilizadas 10% das imagens do *dataset* UFPR-ALPR, que foram separados para essa finalidade durante a fase de treinamento. Essa porção de imagens foi utilizada para testar o desempenho do modelo YOLO e o conjunto dos filtros com o OCR. Cada teste foi realizado de maneira diferente, sendo elas:

- O primeiro teste é realizado com o objetivo de realizar o cálculo das métricas de Recall e Precisão do modelo;

- O segundo teste é executado com o intuito de criar uma função auxiliar que gera um arquivo .csv com a comparação entre as placas reconhecidas e as placas esperadas. Nesse arquivo é possível revisar manualmente os resultados e obter informações mais precisas do reconhecimento, como por exemplo, o cálculo das métricas de precisão e recall do reconhecimento de caracteres.

4.2 Pré-Requisitos dos Testes

Para a realização dos testes, é necessário a disponibilidade de um computador que possua instalado algum ambiente de programação de linguagem Python, e que tenha as bibliotecas OpenCV, Tesseract, TensorFlow e PyTesseract instaladas. Para manter a integridade dos testes é recomendado que a linguagem e as bibliotecas citadas acima sejam instalados nas seguintes versões:

- Python 3.6.5
- OpenCV 4.1.2.30
- Tesseract 0.1.3
- Tensorflow 2.3.1
- PyTesseract 0.3.6

Para realizar a checagem manual dos arquivos .csv, não é necessário nenhuma tecnologia adicional, uma vez que ele pode ser aberto no próprio editor de textos padrão do sistema operacional. No entanto, é recomendado um editor de planilhas, como o Microsoft Excel, para fácil visualização.

4.3 Execução dos Testes

Considerando a existência de dois testes diferentes, um para o detector de placas automotivas e outro para o reconhecimento dos caracteres, os testes são realizados da seguinte maneira.

O teste do detector de placas veiculares YOLOv4 Tiny consiste em executar o sistema de teste de deteção do Darknet, com os pesos treinados (.weights) e as 450 imagens selecionadas do *dataset.* O objetivo do teste é calcular a precisão e recall do modelo baseando-se nos resultados verdadeiros positivos, falsos positivos, verdadeiro negativos e falsos negativos, sendo eles:

- Verdadeiros Positivos: Resultados de detecção correta de placas;
- Falso Positivos: Resultados de detecção incorreta de placas;
- Verdadeiro Negativos: Resultados onde não possui placas nas imagens;
- Falso Negativos: Resultados onde a placa não é detectada, mas existe na figura.

O teste do algoritmo de reconhecimento de caracteres por sua vez consiste em comparar os resultados, após a execução das 450 imagens, a partir do arquivo .csv gerado. Os resultados são verificados manualmente com intuito de classificar verdadeiros positivos, falsos positivos, verdadeiros negativos e falsos negativos, pois com esses dados validados é possível calcular a precisão e recall do reconhecimento. Vale ressaltar que os critérios de avaliação são os mesmos do detector de placas veiculares.

4.4 Resultados

Para apresentar os resultados obtidos, serão utilizados as métricas de precisão e recall, como anteriormente mencionado. Considerando *vp* como verdadeiros positivos, *fp* como falsos positivos, *vn* como verdadeiros negativos e *fn* como falsos negativos, precisão e recall são definidas como nas seguintes equações:

$$precisao \quad deteccao = \frac{vp}{vp + fp} = \frac{placas \quad detectadas \quad corretamente}{todas \quad as \quad placas \quad detectadas}$$

$$recall \quad deteccao = \frac{vp}{vp + fn} = \frac{placas \quad detectadas \quad corretamente}{todas \quad as \quad placas}$$

Dessa forma, a fórmula de *precisao deteccao* é responsável por medir a precisão das predições, ou seja, a porcentagem de predições corretas. E a fórmula de *recall deteccao* mede a capacidade de um modelo de encontrar todos os casos relevantes dentro de um conjunto de dados.

Na Tabela 6 é apresentado os resultados obtidos do modelo treinado YOLOv4 Tiny utilizando o *dataset* UFPR-ALPR.

Tabela 6 - Resultados da detecção de placas veiculares

Conjunto	Precisão	Recall
Treinamento	100%	98,81%
Teste	100%	98,22%

Fonte: Própria

A Tabela 6 acima foi construída da seguinte maneira:

- Para calcular a **precisão de treinamento** do modelo, foi utilizada a fórmula *precisao deteccao*, utilizando os valores 4.002 para placas detectadas corretamente e 4.002 para todas as placas detectadas;

- Para calcular a **precisão de teste** do modelo, foi utilizada a fórmula *precisao deteccao*, utilizando os valores 442 para placas detectadas corretamente e 442 para todas as placas detectadas;
- Para calcular o **recall de treinamento** do modelo, foi utilizada a fórmula *recall deteccao*, utilizando os valores 4.002 para placas detectadas corretamente e 4.050 para todas as placas;
- Para calcular o **recall de teste** do modelo, foi utilizada a fórmula *recall deteccao*, utilizando os valores 442 para placas detectadas corretamente e 450 para todas as placas.

Seguindo os mesmos conceitos das métricas de detecção de placas veiculares, as métricas de reconhecimento de caracteres são definidas pelas equações abaixo:

$$precisao \quad reconhecimento = \frac{vp}{vp + fp} = \frac{caracteres \quad reconhecidos \quad corretamente}{todos \quad os \quad caracteres \quad reconhecidos}$$

$$recall \quad reconhecimento = \frac{vp}{vp + fn} = \frac{caracteres \quad reconhecidos \quad corretamente}{todos \quad os \quad caracteres}$$

De forma explicativa, a fórmula de *precisao reconhecimento* é encarregada por medir a precisão do reconhecimento de caracteres, em outras palavras, a porcentagem de reconhecimentos corretos. Já a fórmula de *recall reconhecimento* mede a capacidade do método de reconhecimento de caracterizar todos os casos de um conjunto de caracteres.

Na Tabela 7 é mostrado os resultados obtidos utilizando o método de reconhecimento desenvolvido nesta monografia, após realização de 16 baterias de testes.

Tabela 7 - Resultados do reconhecimento de placas veiculares

Conjunto	Precisão	Recall
Teste	76,90%	42,06%

Fonte: Própria

A Tabela 7 acima foi criada da seguinte maneira:

- Para calcular a **precisão de teste** do reconhecimento de placas veiculares, foi utilizada a fórmula *precisao reconhecimento*, utilizando os valores 1.325 para caracteres reconhecidos corretamente e 1.723 para todos os caracteres reconhecidos;
- Para calcular a **recall de teste** do reconhecimento de placas veiculares, foi utilizada a fórmula *recall reconhecimento*, utilizando os valores 1.325 para caracteres reconhecidos corretamente e 3.150 para todos os caracteres, no caso 7 x 450 (7 caracteres x 450 placas).

Além dos resultados "gerais" do reconhecimento dos caracteres descritos, na Tabela 8 abaixo é apresentado a quantidade de caracteres específica reconhecida por placa veicular no teste.

Tabela 8 - Relação da quantidade de placas com a quantidade de caracteres reconhecidos

Quantidade de caracteres reconhecidos	Quantidade de placas
Nenhum caractere	80
1	84
2	52
3	39
4	63
5	57
6	42
7	33
Total	450

Fonte: Própria

A seguir é descrito a interpretação da Tabela 8:

- 80 placas veiculares tiveram nenhum caractere reconhecido de forma correta;
- 84 placas veiculares tiveram 1 caractere reconhecido de forma correta;
- 52 placas veiculares tiveram os 2 caracteres reconhecidos de forma correta;
- 39 placas veiculares tiveram os 3 caracteres reconhecidos de forma correta;
- 63 placas veiculares tiveram os 4 caracteres reconhecidos de forma correta;
- 57 placas veiculares tiveram os 5 caracteres reconhecidos de forma correta;
- 42 placas veiculares tiveram os 6 caracteres reconhecidos de forma correta;
- 33 placas veiculares tiveram os 7 caracteres reconhecidos de forma correta.

4.5 Análise e Discussões

Com o intuito de comparar o sistema desenvolvido nesta monografia com trabalhos relacionados, a Tabela 9 apresenta uma comparação do ALPR proposto com o de Laroca et al. (2018) e Laroca et al. (2019). Ambos utilizam YOLO para a detecção de placas e o *dataset* UFPR-ALPR, mas por outro aspecto, Laroca et al. (2018) e Laroca et al. (2019) manipulam uma segunda rede convolucional, a CR-NET, para reconhecimento de caracteres.

Tabela 9 - Resultados obtidos pelo sistema ALPR proposto comparado com o sistema de Laroca et al. (2018) e Laroca et al. (2019) utilizando o *dataset* UFPR-ALPR

ALPR	Precisão de Detecção	Recall de Detecção	Precisão de Reconhecimento	Recall de Reconhecimento
Proposto	100%	98,81%	76,90%	42,06%
Laroca et al. (2018)	-	98,33%	90,37%	-
Laroca et al. (2019)	-	98,67%	82,5%	-

Fonte: Própria

Como mostrado na seção anterior, o sistema proposto obteve resultados significativos na detecção das placas automobilísticas, e como pode ser visto na

Tabela 9, superou o sistema proposto por Laroca et al. (2018) e Laroca et al. (2019) nesse quesito. É notável mencionar que na Tabela 9 o sistema proposto utiliza o YOLOv4 Tiny enquanto os sistemas de Laroca (2018) e Laroca (2019) utilizam respectivamente o YOLOv2 Tiny, e o YOLOv2.

Pode se observar que a comparação dos resultados indica que a utilização de uma segunda rede neural convolucional para tratar do reconhecimento de caracteres se mostra mais efetiva, comparado com a abordagem proposta.

5 Conclusões e Trabalhos Futuros

Esta monografia apresentou um sistema que reconhece placas automobilísticas usando a rede convolucional YOLOv4 Tiny para detecção de placas veiculares, e as bibliotecas OpenCV e Tesseract OCR para o reconhecimento.

O processo de desenvolvimento foi divido em cinco etapas. Na etapa 1 criou-se o modelo de aprendizagem YOLOv4 Tiny. Para isso, usou-se o LabelImg para fazer anotações de 4500 imagens. Além disso, utilizou o framework Darknet para configurar alguns parâmetros da rede. Na etapa 2 a rede convolucional foi treinada com 90% das imagens, e as outras 10% das imagens foram separada para teste. Na etapa 3, utilizou-se o OpenCV e seus filtros para pré-processamento das imagens. A etapa 4, utilizou-se o Tesseract para extrair os caracteres da placa. E ainda, executou-se o pós-processamento para corrigir erros conhecidos no Tesseract. Na etapa 5, realizou-se os testes com os 10% de imagens que fora separado na etapa 2.

O sistema proposto conseguiu atingir o objetivo esperado, uma vez que foi possível fazer o reconhecimento de placas veiculares com sucesso. Por outro lado, os resultados relacionados ao reconhecimento dos caracteres das placas veiculares poderiam ser melhores, caso fossem criados outros filtros para englobar todos os possíveis cenários de imagens do *dataset* UFPR-ALPR.

Com objetivo de validar o sistema, também foi feito um comparativo com trabalhos relacionados, onde o sistema demonstrou superior aos trabalhos propostos por Laroca et al. (2018) e Laroca et al. (2019) com relação a detectar placas veiculares tendo 98,81%, mas inferior no reconhecimento de caracteres, tendo 76,90%.

5.1 Sugestões para Trabalhos Futuros

Como percebido no capítulo 4, existem alguns pontos que podem ser melhorados nos sistema. A seguir, seguem algumas sugestões para trabalhos futuros:

- Criar e implementar um algoritmo que realiza os testes de precisão e recall de maneira automática e autônoma, a partir de um arquivo csv como entrada.

- Implementar uma interface para o sistema, com possível adaptação para dispositivos *mobile*, tornando seu uso mais intuitivo e amigável.

- Criar e implementar uma rede convolucional responsável pelo reconhecimento de caracteres, com objetivo de melhorar a precisão e acurácia.

- Desenvolvimento de novos filtros de pré processamento que melhorem a precisão da detecção do Tesseract.

- Implementação e observação de outras métricas de avaliação de teste.

Referências

ABDULKADER, Ahmad et al. TESSERACT(1) Manual Page. Github, 2020. Disponível em: . Acesso em: 26 de nov. de 2020.

BOCHKOVSKIY, Alexey; WANG, Chien-Yao; LIAO, Hong-Yuan Mark. **YOLOv4: Optimal Speed and Accuracy of Object Detection.** 2020, eprint: 2004.10934.

BRADSKI, Gary; KAEHLER, Adrian. **Learning OpenCV: Computer vision with the OpenCV library**. 1. ed. 1005 Gravenstein Highway North, Sebastopol, CA 95472: O'Reilly Media, Inc., 2008. 1026 p. ISBN 978-0-596-51613-0.

BRASIL. Resolução nº 780, de 26 de Junho de 2019. Dispõe sobre o novo sistema de Placas de Identificação Veicular. **Ministério da Infraestrutura/Conselho Nacional de Trânsito**, 2019. Disponível em: <http://www.in.gov.br/web/dou/-/resolucao-n-780-de-26-de-junho-de-2019-179414765>. Acesso em: 24 de maio de 2020.

CAMPOS, Teofilo Emidio de. **Técnicas de Seleção de Atributos e de Classificação para Reconhecimento de Faces**. 2000. Disponível em: <http://www.vision.ime.usp.br/~teo/publications/qualificacao/quali.html>. Acesso em 6 de jun. de 2020.

CASTRO, Mariangela de. Número de multas de trânsito volta a aumentar na capital paulista. **Folha de São Paulo**, São Paulo, p. 1-1, 6 de dez. de 2019. Disponível em: <https://agora.folha.uol.com.br/sao-paulo/2019/12/numero-de-multas-de-transito-volta-a-aumentar-na-capital-paulista.shtml>. Acesso em: 7 de jun. de 2020.

CLEMENTE, Bruno; RODRIGUEZ, Roberto; GAY, Antonio. Técnicas de Segmentação de Imagens, Extração de Características e Reconhecimento de Caracteres de Placas de Veículos. Universidade Federal do Rio de Janeiro, Área de Ensino e Pesquisa, NCE/IM, 2002. Disponível em: <http://www.nce.ufrj.br/labic/downloads/7spuc-rs2002.pdf>. Acesso em 6 de jun. de 2020.

Dúvidas Frequentes. DETRAN, [s.d.]. Disponível em: < https://www.detran.sp.gov.br/wps/portal/portaldetran/cidadao/duvidasFrequentes/sa-veiculos/sa-emplacamento/sa-detalhesemplacamento/cd3fc1ed-ba26-4005-82f5-5b5534ac1be4/!ut/p/z0/jU7LTsMwEPwWDj1au4nj0h4NJ1p6AqHgC9o4Tmvq2nk4Af4eEyRUJIR6Ws1jZwYUlKA8TXZP0QZPLuFntXzhBT6sbzKJuHnkKFfX26fsXvK7NcIG1P-GlGBfu05JUDr4aN4jlG3oI7naxJ78An_uONmahqY33WiSc1jgQGwyVo8ufANzah1pOiU5zET6JXcww29B17zRmalZRfmSFYiCrfJGMFEJwQvSWWWKr2F5v7vd7UG1FA_M-iZAedY4g_PgmfirEcqLGtujqj7e5NUn_QNArQ!!/>. Acesso em: 01 de maio de 2020.

FORSYTH, David A.; PONCE, Jean. **Computer vision: a modern approach**. 2. ed. [S. l.]: Prentice Hall Professional Technical Reference, 2002. 720 p. ISBN 978-0-13-085198-7.

FUJITA, Hamido; U, Raghavendra; BHANDARY, Sulatha; GUDIGAR, Anjan; TAN, Jen Hong; ACHARYA, Rajendra. **Deep Convolution Neural Network for Accurate Diagnosis of Glaucoma Using Digital Fundus Images**. Information Sciences, [*S. l.*], v. 441, p. 441, 1 fev. 2018. DOI 10.1016/j.ins.2018.01.051. Disponível em: https://www.researchgate.net/publication/322892996_Deep_Convolution_Neural_Network_for_Accurate_Diagnosis_of_Glaucoma_Using_Digital_Fundus_Images. Acesso em: 18 nov. 2020.

GAD, Ahmed Fawzy; JOHN, Suresh. **Practical computer vision applications using deep learning with CNNs**. 1. ed. Menoufia, Egypt: Apress, 2018. 405 p. ISBN 978-1-4842-4167-7.

GJORESK, Martin. Tesseract OCR engine architecture. ResearchGate, 2014. Disponível em: <https://www.researchgate.net/figure/Tesseract-OCR-engine-architecture_fig4_265087843>. Acesso em: 31 de maio de 2020.

GONÇALVES, Gabriel Resende; MENOTTI, David; SCHWARTZ, William Robson. License Plate Recognition based on Temporal Redundancy. Universidade Federal de

Minas Gerais, 2016. Disponível em: <https://homepages.dcc.ufmg.br/~william/papers/paper_2016_ITSC.pdf>. Acesso em: 6 de jun. de 2020.

GONZALEZ, Rafael; WOODS, Richard E. Digital Image Processing: Computer Vision, Graphics, and Image Processing. 3rd. ed. [S. l.: s. n.], 2007. 954 p. ISBN 013168728X.

HIJAZI, Samer *et al*. Using Convolutional Neural Networks for Image Recognition. **Cadence Design Systems**, San Jose, CA, USA, p. 1-12, 1 jan. 2015. Disponível em: http://site.eet-china.com/webinar/pdf/Cadence_0425_webinar_WP.pdf. Acesso em: 22 nov. 2020.

HUNG, V. Tensorflow-yolov4-tflite. Github, 2020. Disponível em: <https://github.com/hunglc007/tensorflow-yolov4-tflite>. Acesso em: 28 de out. de 2020.

Introduction. OpenCV, 2020. Disponível em: <https://docs.opencv.org/master/d1/dfb/intro.html>. Acesso em: 16 de novembro de 2020.

IOFFE, Sergey; SZEGEDY, Christian. Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift. **Proceedings of the 32nd International Conference on Machine Learning**, [*S. l.*], v. 37, p. 448--456, 9 jul. 2015. Disponível em: http://proceedings.mlr.press/v37/ioffe15.html. Acesso em: 22 nov. 2020.

KRIZHEVSKY, Alex *et al*. ImageNet Classification with Deep Convolutional Neural Networks. **Advances in Neural Information Processing Systems**, Curran Associates, Inc., v. 25, p. 1097--1105, 1 jan. 2012. Disponível em: https://proceedings.neurips.cc/paper/2012/file/c399862d3b9d6b76c8436e924a68c45b-Paper.pdf. Acesso em: 22 nov. 2020.

LAROCA, Rayson et. **A Robust Real-Time Automatic License Plate Recognition Based on the YOLO Detector,** 2018 International Joint Conference on Neural

Networks (IJCNN), Rio de Janeiro, 2018, pp. 1-10, doi: 10.1109/IJCNN.2018.8489629.

LAROCA, Rayson et. “**An Efficient and Layout-Independent Automatic License Plate Recognition System Based on the YOLO Detector**,” arXiv preprint arXiv:1909.01754, 2019.

Learning. 2015. Disponível em: <https://www.researchgate.net/publication/309282609_Brazilian_License_Plate_Character_Recognition_using_Deep_Learning >. Acesso em: 6 de jun. De 2020.

LECUN, Yann; BOTTOU, Leon; BENGIO, Yoshua; HAFFNER, Patrick. Gradient-based learning applied to document recognition. **Proceedings of the IEEE**, [*S. l.*], v. 86, p. 2278 - 2324, 1 nov. 1998. DOI 10.1109/5.726791. Disponível em: https://ieeexplore.ieee.org/document/726791. Acesso em: 18 nov. 2020.

LECUN, Yann; BENGIO, Y.; HINTON, Geoffrey. Deep Learning. **Nature**, [*S. l.*], v. 521, p. 436-44, 1 maio 2015. DOI 10.1038/nature14539. Disponível em: https://www.researchgate.net/publication/277411157_Deep_Learning. Acesso em: 21 nov. 2020.

MITHE, Ravina; INDALKAR, Supriya; DIVEKAR, Nilam. Optical character recognition. **International journal of recent technology and engineering (IJRTE)**, v. 2, n. 1, p. 72-75, 2013.

MOHAMAD, Mustafa. 3D Object Recognition using Local Shape Descriptors. **Technical Report**, no. 2013-614. 2013.

MOREIRA, I. de C. A inclusão social e a popularização da ciência e da tecnologia no Brasil. Inclusão Social, Brasília, v. 1, n. 2, p. 11-16, abr./set., 2006.

PEIXOTO, S; GONÇALVES, Gabriel; CHÁVEZ, Guillermo; MENOTTI, David; SCHWARTZ, William. Brazilian License Plate Character Recognition using Deep Learning, Workshop de Visão Computacional - WVC 201. 2015

LIN. Tzuta. Labelimg. 2015. Disponível em: https://github.com/tzutalin/labelImg. Acesso em: 18 dez. 2020.

Placas de veículos no Brasil cores. Wikipédia, 2010. Disponível em: <https://pt.m.wikipedia.org/wiki/Ficheiro:Placas_de_ve%C3%ADculos_no_Brasil_cores.png>. Acesso em: 24 de maio de 2020.

REDMON, Joseph. **Darknet: Open Source Neural Networks in C**. [*S. l.*], 2013-2016. Disponível em: https://pjreddie.com/darknet/. Acesso em: 20 nov. 2020.

REDMON, Joseph. et al. **You only look once: Unified, real-time object detection**. CoRR, abs/1506.02640, 2015. Disponível em: <http://arxiv.org/abs/1506.02640>. Acesso em: 20 nov. 2020.

REDMON, Joseph; FARHADI, Ali. **YOLO9000: Better, Faster, Stronger**. [*S. l.*]: ArXiv preprint arXiv:1612.08242, 2016. Disponível em: https://arxiv.org/abs/1612.08242. Acesso em: 20 nov. 2020.

REDMON, Joseph; FARHADI, Ali. **YOLOv3: An Incremental Improvement**. [*S. l.*]: arXiv, 2018. Disponível em: https://pjreddie.com/media/files/papers/YOLOv3.pdf. Acesso em: 20 nov. 2020.

RUSS, John C. **Computer-Assisted Microscopy**: The Measurement and Analysis of Images. 1. ed. Plenum Press, New York: Springer US, 1990. 470 p. ISBN 978-1-4613-0563-7.

SMITH, Ray. An Overview of the Tesseract OCR Engine. Google Inc. 2007.

SOUTO-MAIOR, Cesar Duarte; SANTA, Stephane Louise Boca. Consumo Sustentável: Ações para redução do consumo de papel em um banco regional de Santa Catarina. **RMGC – Revista Metropolitana de Governança Corporativa**, [*S. l.*], p. 37-49, 27 out. 2018.

SUZUKI, Satoshi; ABE, Keiichi. **Topological structural analysis of digitized binary images by border following**: Computer Vision, Graphics, and Image Processing. [*S. l.*: *s. n.*], 1985. v. 30. ISBN 0734-189X. DOI https://doi.org/10.1016/0734-189X(85)90016-7. Disponível em: http://www.sciencedirect.com/science/article/pii/0734189X85900167. Acesso em: 17 dez. 2020.

SZEGEDY, Christian *et al*. Going deeper with convolutions. **2015 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)**, [*S. l.*], p. 1-9, 7 jun. 2015. DOI 10.1109/CVPR.2015.7298594. Disponível em: https://ieeexplore.ieee.org/document/7298594. Acesso em: 20 nov. 2020.

UFPR-ALPR Dataset. Universidade Federal do Paraná, 2018. Disponível em: https://web.inf.ufpr.br/vri/databases/ufpr-alpr/. Acesso em: 17 nov. 2020.

USANDO o formato SavedModel. [*S. l.*], 2020. Disponível em: https://www.tensorflow.org/guide/saved_model. Acesso em: 22 nov. 2020.

VARGAS, Ana Caroline Gomes; PAES, Aline; VASCONCELOS, Cristina Nader. Um estudo sobre redes neurais convolucionais e sua aplicação em detecção de pedestres. In: Proceedings of the XXIX Conference on Graphics, Patterns and Images. 2016. p. 1-4.

VARTY, Josh. Shotgun Debugging. **Joshvarty**, [*S. l.*], p. 1, 4 fev. 2018. Disponível em: https://joshvarty.com/2018/02/04/ltfn-4-intro-to-convolutional-neural-networks/. Acesso em: 21 nov. 2020.

WEI, Wei *et al*. Accurate and Fast URL Phishing Detector: A Convolutional Neural Network Approach. **Computer Networks**, [*S. l.*], v. 178, p. 107275, 1 abr. 2020. DOI 10.1016/j.comnet.2020.107275. Disponível em: https://www.researchgate.net/publication/340973351_Accurate_and_Fast_URL_Phishing_Detector_A_Convolutional_Neural_Network_Approach. Acesso em: 22 nov. 2020.

YOLO v4, v3 and v2 for Windows and Linux: neural networks for object detection. Github, 2020. Disponível em: https://github.com/AlexeyAB/darknet. Acesso em: 22 nov. 2020.